SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 9 - Lecture Transcripts

Lectures 115-125 | 11 lectures | 94 pages (source transcript pages 967-1059)

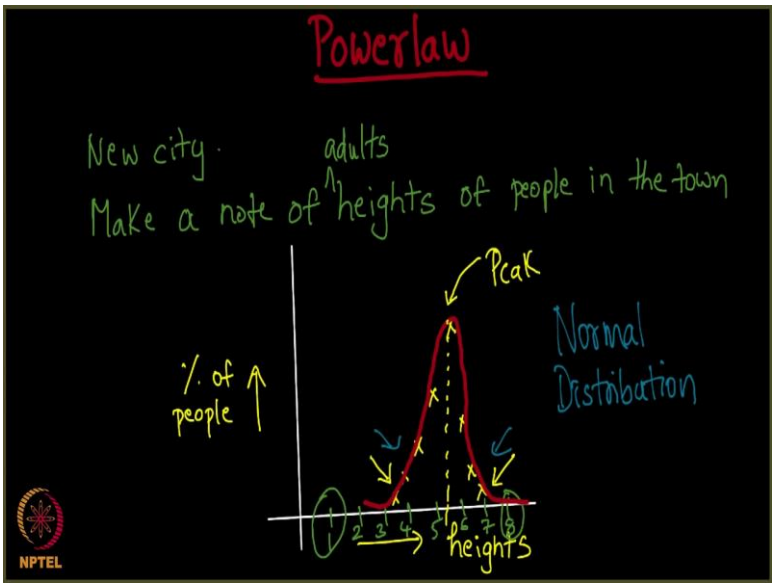
INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L115	Introduction to Power Law	2
L116	Why Do Normal Distributions Appear?	9
L117	Power Law Emerges in WWW Graphs	14
L118	Detecting the Presence of Power Law	21
L119	Rich Get Richer Phenomenon	26
L120	Summary So Far	33
L121	Implementing Rich-Getting-Richer Phenomenon (Barabasi-Albert Model)-1	36
L122	Implementing Rich-Getting-Richer Phenomenon (Barabasi-Albert Model)-2	42
L123	Implementing a Random Graph (Erdos-Renyi Model)-1	68
L124	Implementing a Random Graph (Erdos-Renyi Model)-2	72
L125	Forced Versus Random Removal of Nodes (Attack Survivability)	82

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Rich Get Richer Phenomenon

Lecture – 115
Introduction to Power law

(Refer Slide Time: 00:08)



So, we are now starting a brand-new chapter called the Power Law. So, it is difficult to; difficult on your minds if I start talking about power law directly. So, as and always it has been customary, that we start with an example right with a nice story rather ok. So, what I will do is I will now ask you all this question. Assume if I go to a new city and I make a note of everybody's heights of everyone in the town. I make a note of every ones height of people in the town. I come back and then I look at this data. And I try plotting the following very simple graph. The graph will be this all my x axis, I will have heights; on my y axis I will have percentage of people with that height.

Now, let me illustrate this with a numeric value. So, what I do is let us say this is 1, this is 2, this is 3; 1, 2, 3, 4, 5, 6, 7. So, I forgot to mention that I am making note of only height of adults in the town ok. As you would have guessed they there is no reason why we should take one foot 1 foot or let us say 8 and above, we do not know of anyone whose of this height adults especially.

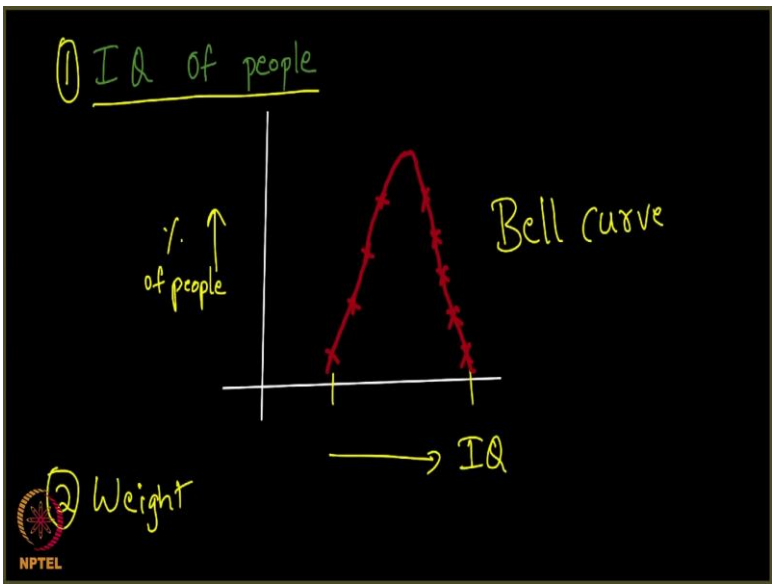
So, what I do is this plot mainly goes like this. I look at the percentage of people who are 5 feet tall, percentage of people who are 6 feet tall. And most of the people as you would have guessed will be between 5 and 6. So, around 5.3, 5.4 will be the peak. There will be very less people with 6 and over very very few people who will be 7 feet tall, very unlikely in many countries although you will find tall people; it is very country specific. You might for find all people in different countries, but most the countries will not have very tall people above 7 feet.

So, they are going to be very very less. And people just above four are going to be less; people who are 4 feet are very very less. And this is how your plot will look like as you would have guessed ok. It is it goes up and comes down like this. So, what do we observe? We observe that there are very few people with very less height. And very few people with who are very very tall and most the people are between this ok.

Let us say between 5 and 6 feet correct. This is varied peaks; such a distribution is called a normal distribution; this is called a normal distribution. What do you? What do you mean by normal distribution? The distribution where it is less on the left and right, and it peaks somewhere in the middle right ok.

So, we can see many things in life which has normal distribution things.

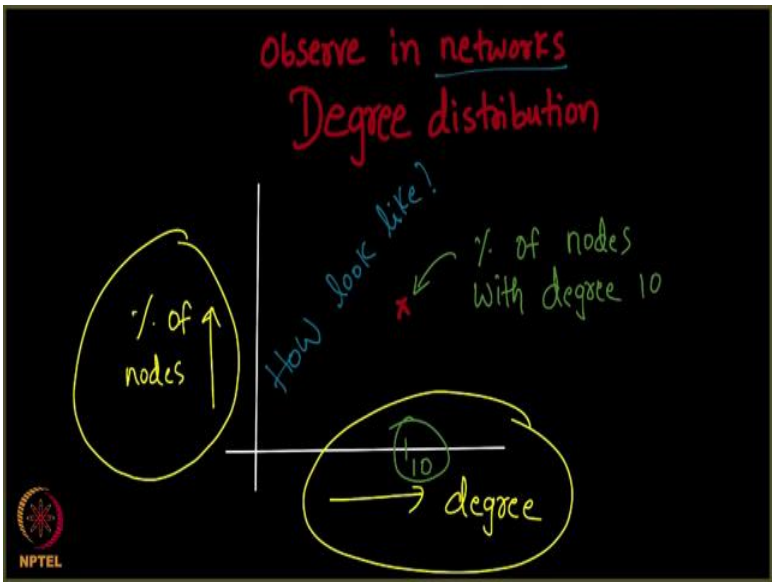
(Refer Slide Time: 03:57)



For example, let us say IQ of people; IQ of people in a classroom; you will see you will see if I were to plot on the x and y axis. As and always as I told you this is going to be my in this example IQ of people. And y axis will be percentage of people with that IQ right. So, let us say this is slightly on the lower side; this is slightly on the higher side; you will find very few people with very very low IQ; meaning below average very few people with extremely high IQ. And most of them will sort of lie in the middle, and if you actually try plotting values for everything; you will indeed get a curve like this. And this curve in statistics is called the Bell curve. It resembles the shape of the bell that's why it is called the bell curve. In fact, it is easy to capture it mathematically as well although we will not discuss it right now.

So, IQ of people forms a bell curve you can think of many such examples right; weight of people in a town, weight is another such example, IQ being one such example, height being another example right. So, let us look at an example which is very specific to our work to our subject and that is going to be next.

(Refer Slide Time: 05:35)

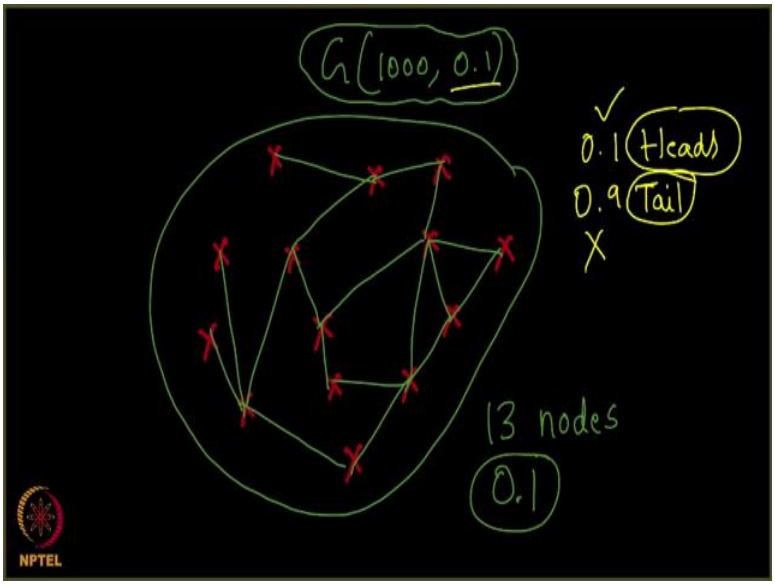


So, if I what do we observe, what do we observe? If we were to take a network and try to ask the degree distribution in the network; degree distribution what do you even mean by this? By degree distribution I mean again x and y axis ok. This is my x axis and my x axis. How long my x axis will be degree of nodes. And along my y axis will be percentage of nodes with that degree; with that I mean if you have something like 10

here; just this point will denote the let me just write the downs its seems into your minds. This denotes the percentage of nodes with degree 10, degree 10.

And big question right now is, how does this plot look like? How does this plot look like? For that we should first take a network, now here is a good the chance for you all to write a piece of code and observe the following.

(Refer Slide Time: 07:03)



Let me take you all should probably consider a graph with 1000 nodes. And with probability let us say point 1 you put edges between them. What do you mean by this? By this you mean let us say we had 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 nodes something like this.

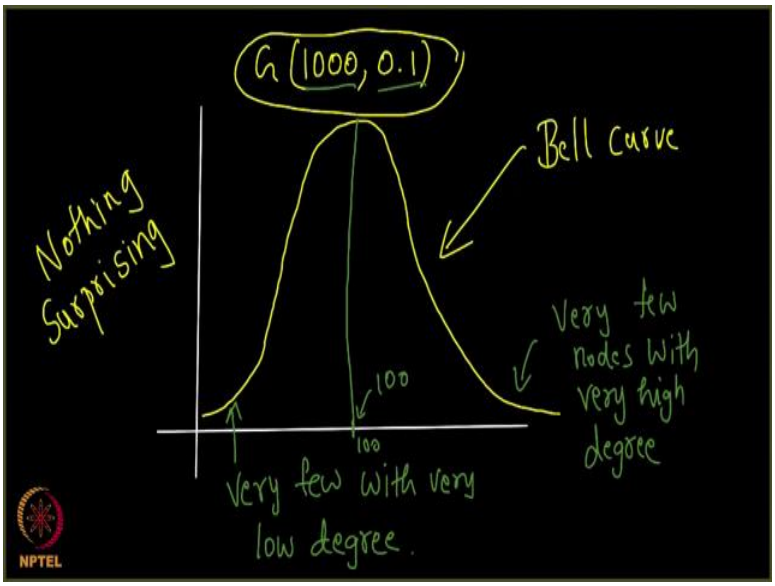
What you do is you will put edges by considering the probability 0.1. What do what do I mean by probability 0.1? We have discussed the set length before, but I will just recalculate. You toss a coin with head and tail probability 0.1; heads probability and 0.9 will be your tail probability right.

If you get a head, you put an edge; if you get a tail, you do not put an edge that is how I am continuing this right. Assume I got a network something like this right. This will be my final network. And this is my graph G with probability, how many how many nodes are here? 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 nodes, this is graph 13 nodes and edge

probability is 0.1 which means I toss a coin with probability of it being heads is 0.1 and tail being 0.9, I put an edge if I get a head I do not put an edge; if it is a tail.

I repeat this process for every possible edge on this 13 node graph ok. I showed you a simple example, but what you should be doing is do it for a 1000 node graph right.

(Refer Slide Time: 08:56) .



Do it for a 1000 node graph. Once you finish programming, you get 1000 node graph. And put edges with probability 0.1. This is called a G (1000, 0.1). And what should you do? As you would have guessed compute it is degree distribution. We discussed in our, previous slide; what is the degree distribution. You put degree on the x axis and you put the percentage of nodes on the y axis, and see how this plots looks like.

You will be started to see, that this plot will resemble a bell curve once again with very few nodes, very few nodes with very high degree with very high degree. At the same time, you will see very few nodes, very few nodes with very low degree. And again, it will peak somewhere in the middle.

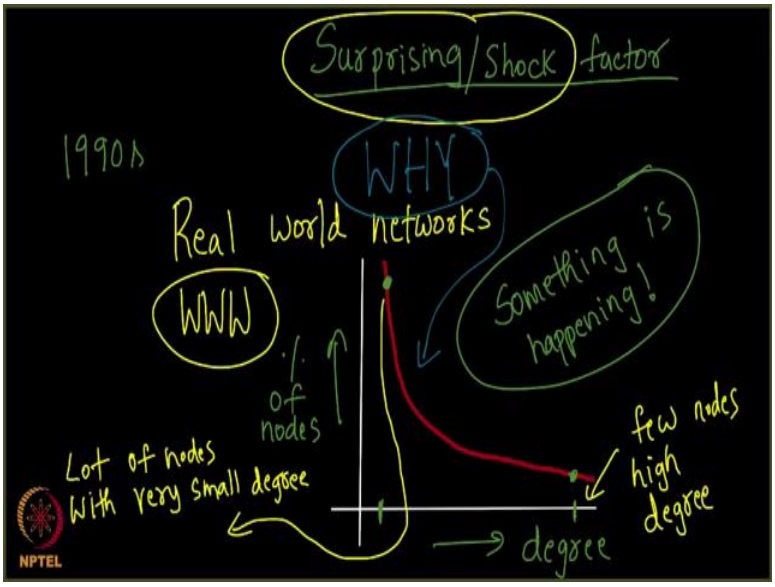
If you can guess, it peaks around 100 for x being 100. It peaks you can probably tell me why it is the product of 1000 and 0.1. It is lift as a it is left as a quiz for you all to say why does it peak at 100. Anyways, the moral the story is simple. When we draw out a graph with 1000 nodes and 0.1 edge probability, we will see again a bell curve. You

probably you would tell me yeah, we saw it in the previous cases, we saw many examples right it is not at all surprising for us right.

So, IQ of people you saw, you saw, that the degree of nodes follow. What is called the normal distribution? And everything else looks similar right. If you look at weights of people, height of people, IQ of people all of them seem to follow this bell curve only nothing surprising about it right. So, let me write that down there is nothing surprising it nothing surprising.

Then in the late 1990s scientist observed something that is actually very surprising.

(Refer Slide Time: 11:31)



So, what do you mean by the very word surprise? Surprise is something that happens unexpectedly. You think that this is how it should be, but nature has something else in store. You see something else happening where what you thought should have happened is not what is happening there. And that is what creates a surprising or what is called a shock factor. In the late 1990's network scientist observed a very nice phenomenon. They took at the, they looked at several real-world networks. So, until 1990's people did not have big networks. That was because we did not have computational ability back then. It is only in the 1880's and 1990's that we started collecting a lot of data and we had data sets we had computers which would crunch the data sets.

So, let us say they if you consider the network of the World Wide Web. It is a few millions of nodes. People ask the same question. What will be the distribution of degree on these networks? And they all expected it will be a bell curve. And as you would have guessed, there was a surprise stroke shock factor there and the fact was the following. They observed that it was no way close to the bell curve. The curve looked something like this.

There was a drop; it was not a bell like this. It was not like this; it was a drop like this. So, once again what is the x and y axis? x axis is the degree and y axis is the percentage of nodes that have so much degree. So, people observed that there is a drop here. Why is it happening right? So, if let us say you try drinking milk tomorrow morning and milk taste salty let us say; you will actually start wondering what went wrong, why is it why is my milk tasting, salty of all things right; something should I gone wrong right, something should be there is some background information that you are missing.

So, similarly when scientist observed that drop of the curve like this which is nowhere close to the bell curve, people thought something must be happening here that we do not seem to understand. So, let we write that now something is happening here that we do not understand. So, what could be happening why exactly is this observed. What is this even mean?

You see there are a lot of nodes with very low degree and very few nodes right; very few nodes with very high degree that is what it means right. Few nodes with high degree and then let me write this here lot of nodes with very small degree. And this startle the community of network scientist. They started wondering why this a property is being seen, what is making this property emerge.

Now, is the right time for you all to pause the video and think about the question. Why is this happening the big why? Why is it that we are not able to see a normal distribution here? Why are we seeing a drop in the curve like this?

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Rich Get Richer Phenomenon

Lecture – 116
Why do Normal Distributions Appear?

(Refer Slide Time: 00:11)



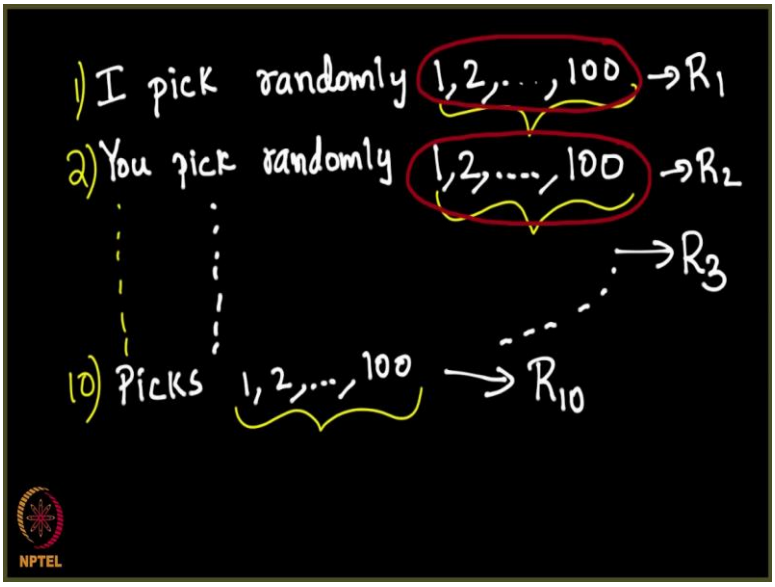
So, we saw two things. One was the normal distribution right. We saw basically two plots. So, far one is that I told you people there is this normal distribution, the inverted curve and then a drop like this ok. So, the definition the example heights of people in a in a town and this happens to be the degree distribution of nodes in a real-world network you know real world; we see this ok.

Now, before asking why do we see a very different kind of a plot here? Why in most of the situations, we see this kind of a plot. Before asking this question, we would like to ask individually, why and how does this come about? This one same time why and how of this as well.

So, to begin with let me ask the why and how of this normal distribution. How does normal distribution come by ok? Let us take a look at a nice example. And try to motivate the reason behind why the normal distribution appears in nature ok. Here is an

example you should invite you all to think a little about it ok. It is not very difficult to understand it is pretty straight forward ok.

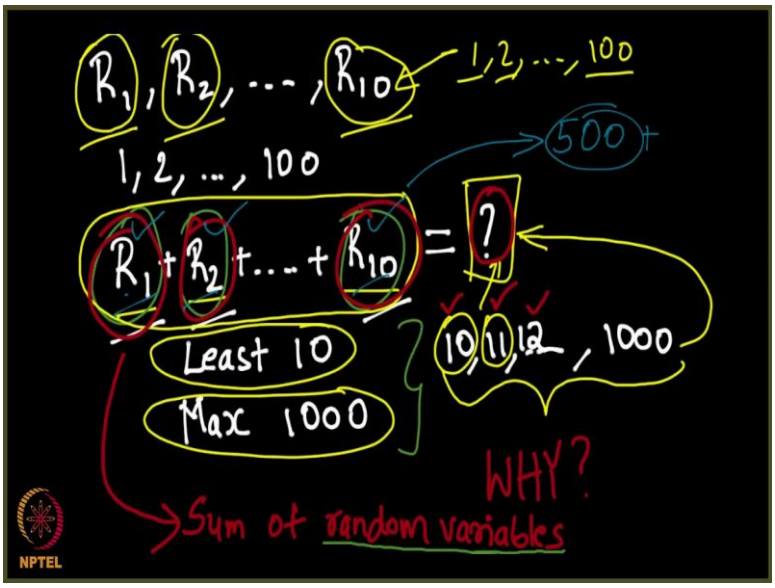
(Refer Slide Time: 02:01)



So, assume I pick a random number; randomly I pick randomly a number from 1 to 100. One of these numbers I pick someone number ok. And then you as another person, you also pick randomly some number from again from the same set 1 to 100. Some number you will pick randomly here all right good. Now see when I pick a number randomly here from 1 to 100, I can pick any number; it could be a 1 it could be a 2, it could be a 3, it could be a 50, 60, 73 89 100. Whatever I want when you pick you as well will have the liberty to pick any number.

Let us assume 10 such people pick. I am the first person; you are second person so on and so forth; some 10 people pick like this ok. Tenth person also picks a number from 1, 2, 100 right. So, what I, now what I am now going to do is let us say the number here that I pick let me call it R_1. The number you pick I call it R_2, the third person picks a number I am going to call it R_3 and so on. And the 10 person who picks the number that number is R_{10} ok.

(Refer Slide Time: 03:52)

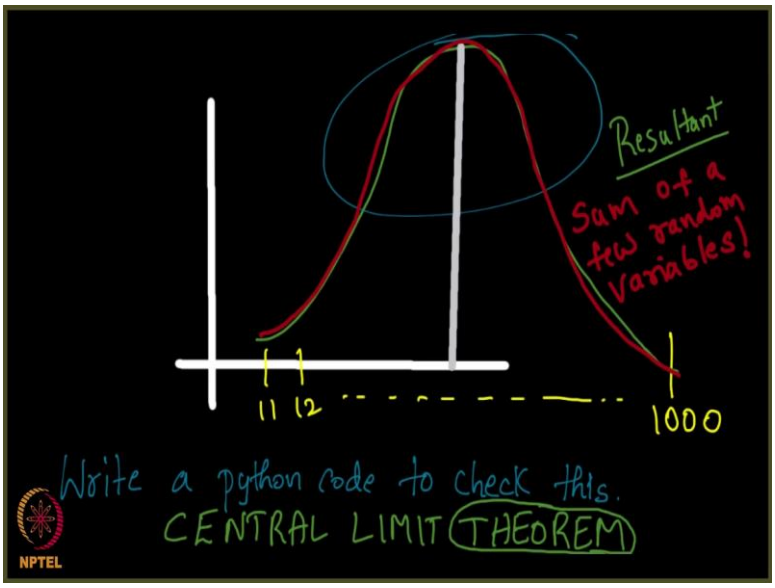


So, let me go to the next slide, let me write down. There are 10 random numbers each one between 1, 2, and 100. Each of these random numbers are from 1, 2, 100. Here is an important question while each of these random numbers can be anything from 1, to 100; just in case I added them look at this just in case I added all these, what will this be? See each one of these can be a number from 1 to 100 which means the sum will be at the least it will be 1 plus 1 plus 1 10 times. Assume all them shows a 1, 1, 1 each then at least it will be 10 and a maximum of how much.

If each one of them picked 100 it will be 100, 100, 100, 10 times. It will be 1000. So, this is the least and this is the maximum the minimum and maximum, but then, but then observe here that any number 10, 11, 12 up to 1000 can appear for what for the sum correct, but here is the climax. While whatever numbers you pick can be any number between 1, 2, and 100 when you take these ten numbers and add them, what you get as an answer is not uniformly at random from these numbers which means 10 and 11 do not appear uniformly at random here. While let us say in R_1 and 2 and 3 and what not up to 100 appears randomly. Each one of them is has the same chance of appearing here, but then the chances of 10 appearing here is less than the chances of 11 appearing here is less than the chances of let us say 12 appearing here, here in this place. And why is that?

If you can think; you will conclude that this is indeed once again a bell curve right.

(Refer Slide Time: 06:25)



It is going to be if I were to plot the values; let us say the least what 11 12 so, on up to it will go on and on and up to let us say 1000 is the maximum. You will observe that uniform. So, let me go back to the previous slide. So, each one of them each one of R 1 can be on an average let us say anything from 1 to 100 so on and so forth. So, this will be roughly; so, this will be like this the plot will look like this right. It is good to write a programming and then check.

Let me assign that to you the assignment to you all to write is to write, a piece of python code; write a python code to check this. You will be surprised to see that it is actually a very beautiful curve like this ok, to check this all right. So, why it is this happen that is very obvious. It is very unlikely that you pick a 1 and a 1 and a 1 for all of them correct. It is equally unlikely that you pick a 100 and a 100 and a 100 for all of them correct.

So, what is more probable as a sum i if you if you have guessed it right it is roughly around 500, 550 but roughly around 500 plus right. That will go, that will be free; that is the peak here, rapped that the peak here is 550 and completely the peak here is going to be 550 that is the peak ok. Anyway, you do not worry much about it. All you need to know is that you will see a normal distribution here. Now why is that? Why does it why what exactly is the moral of the story?

The moral of the story is the sum of random variables that you see here. The sum of random variables they are basically a bunch of a sum entities that are random. They are

called the random variables, sum of random variables. And this is slightly it is the term that would not have really heard of well unless you have done a course in statistics, but just see random variables as simply random entities which can take any values all right. Now whenever, the resultant this; a resultant whenever the resultant is a sum of several random variables; whenever the resultant is a sum of a few random variables several few random entities random variables, then you will see a normal distribution like this.

You will see this distribution whenever the resultant is a sum of few random variables. This goes by the name this is actually a very well-known theorem in statistics rather probability. It is called the of central limit theorem. So, I have stated in a very in a really easiest possible way, you do not worry much about the word theorem here. It is sort of scary for many of you people who do not like math much, but the all I am saying here is that the sum of random variables right.

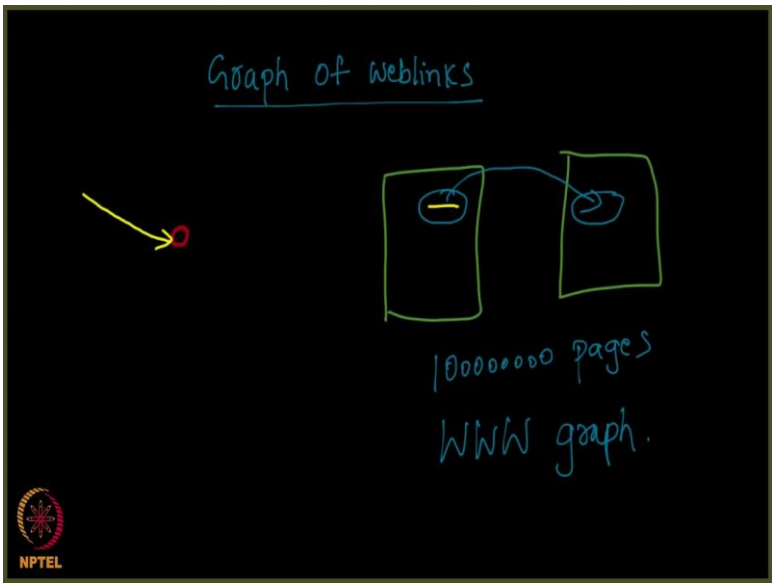
This sum of random variables is this resultant and that resultant is generally a bell curve if its sum of few random variables. And that is how, this emerges. This always emerges when the resultant is a sum of random variables this. In fact, true in nature whenever you see an experiment. The sum will basically the resultant that you are plotting will basically be a bunch of random events put together right ok.

So, let me go to the new slide and let me ask the next question that the next question is in the first slide itself. that I question is this. How does this emerge? What is the reason behind the emergence of this thing? We gave some reason for the left plot this one. So, what is the reason for this one is what we will be covering in the next lecture.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Rich Get Richer Phenomenon

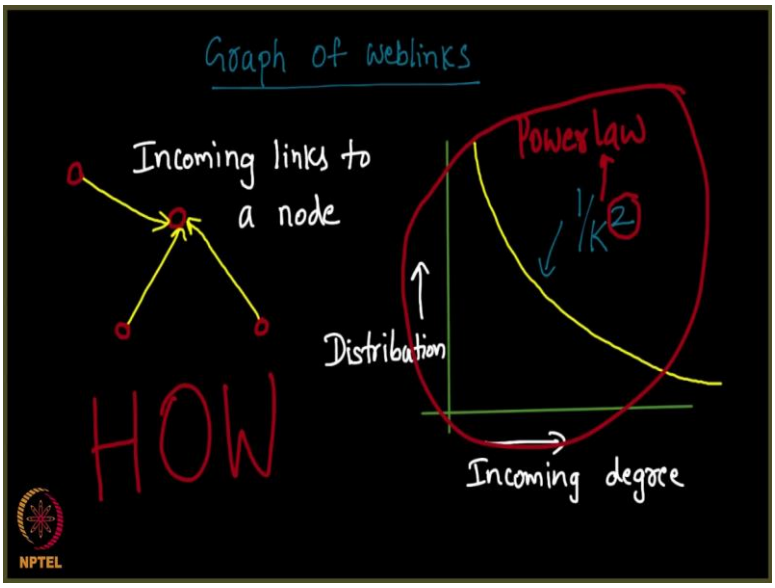
Lecture – 117
Power Law emerges in WWW graphs

(Refer Slide Time: 00:05)



So, remember that the graphs that we discussed before. If you do not remember, let me revise it quickly for you all. A web graph is basically a graph with a node ok. This node represents let us say a page like this a page pointing to another page right. So, the here is a page which has a hyperlink to another page. You click on this, you are taken to a new page right and I and internet has millions and millions of such pages right; several such a page, a whole lot of such pages correct. And then I look at the underlying graph and that is called the web graph, the graph of the World Wide Web.

(Refer Slide Time: 01:03)

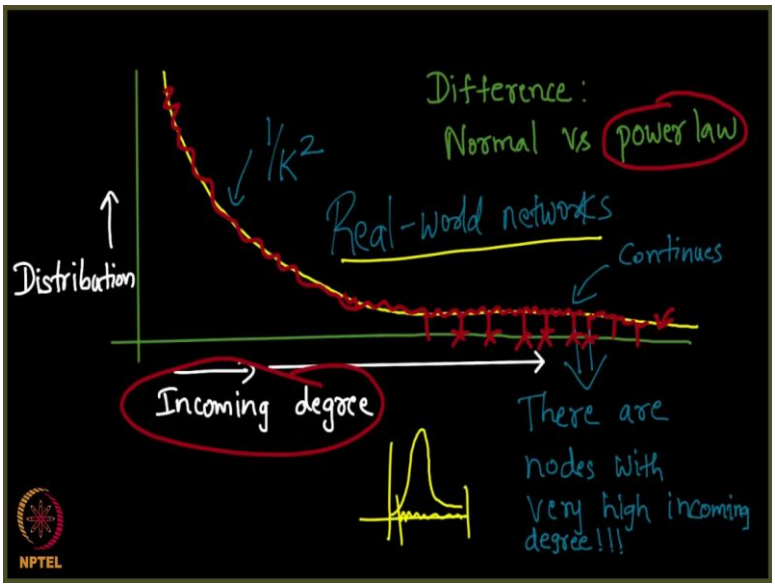


So, I am going to consider that right now ok. So, typically a node represents a node represents a page, this represents a page. And an incoming link to it represents a link that is pointing to this page as simple as that right ok. So, there may be many such incoming links to a webpage from some other page. By that time in if I put a link to your home page from my home page then, I put an edge from my home page to your home page. Home pages are all nodes here and if there is a link from one page to the other, you put a directed edge right. These are all the web pages from where there is a link put to this page.

So, these are the incoming links to a given right. Here the incoming link is incoming degree is 3 right; that is called the degree. Now, let me plot on x and y axis the following. On x axis, I will write the incoming degree and on y axis I will draw the distribution right and I will observe what is happening here. I observe that there is a drop like this. Unlike in the previous case, we do not see a normal distribution here and in fact, a closer observation even tells me that this plot very much resembles $1/K^2$ where K is the x axis. It resembles this right. And this is called the power law right; the law where the, the law which states that some distributions do not exhibit normal behaviour. You do not see a bell curve there; you see a drop like this ok.

So, the exponent here is to, it can be anything for what we now it can be anything. But in this case in the case of the web links, it turns out to be K square ok.

(Refer Slide Time: 03:09)



So, what are we observed so far? When you plotted the incoming degree versus the distribution right, you got this for web graphs. But then when you would you this is 1 over K square, but then there is a difference. What is the difference? Normal versus power law.

What is the difference? See, initially you saw we plotted two graphs. One was normal, one was power law right. What is the huge difference between these two things? What does power law tell us which normal distribution did not? We see this in fact, this actually extends right. The this is the yellow line here, let me show that you once again; this actually extends, it goes beyond; for a for a long time you will see a curve like this ok.

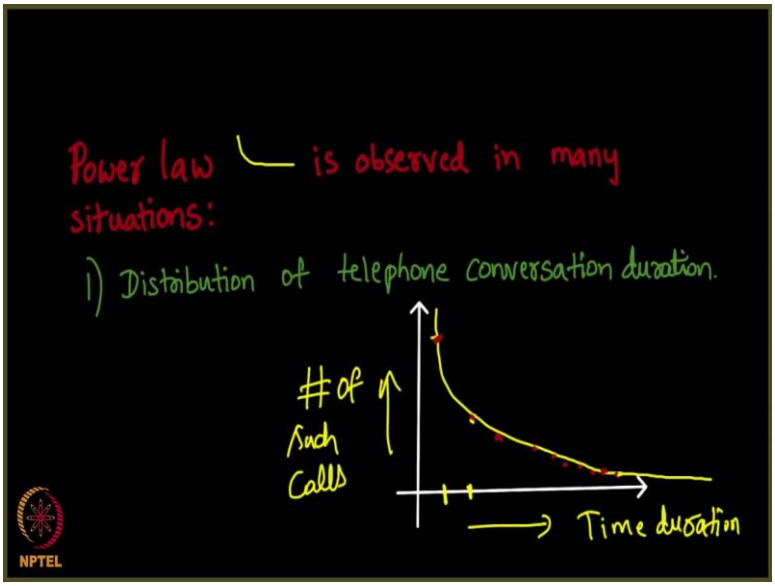
So, let me just extend this green line also. And what is this signify? What is this band here? What is this band here? What is it denote? Please look at the x and y axis, its rather self explanatory. What does it denote? It denotes something very significant, observe carefully. It denotes that this continues this goes on; goes on means what? It means, there are nodes with very high incoming degree Do not you think that is what this means? Look at this side this x axis, x axis means what?

We just note it, the incoming degree. The incoming degree is very high here as it goes on the right side of the x axis and the yellow line denotes that there are nodes; there are nodes with very high degree. That was not the case in normal distribution as you

remember correct. The power laws simply say that the curve goes a extends beyond a stage. And it sort of is close to the x axis although it does not become equal to x axis; it goes on which means the it shows of shows us of the existence of nodes that have very high incoming degree right, fine. Here is the question. Why do you observe this drop of the curve? And secondly, how come there are so many nodes with higher and higher degree is observed in real world networks?

I hope you remember the in the previous lecture. One of the previous two lectures, we observed that if you plot it for let us say random graph which is a GNP model, with some 1000 nodes and probability 0.1. You observe that it was a quick drop. How was the graph? Remember the graph was something like this that is all. After this there was no extension, before this there is no value no node which satisfies this. It is all it all lies between this and this, but in case of real-world networks we see an its sort of extends. Firstly, it drops it drops and extends right. Both funny and strange; why is this happening will be the crux of this chapter; and the crux of discussions in the forthcoming lectures. Let us go ahead ok.

(Refer Slide Time: 06:39).



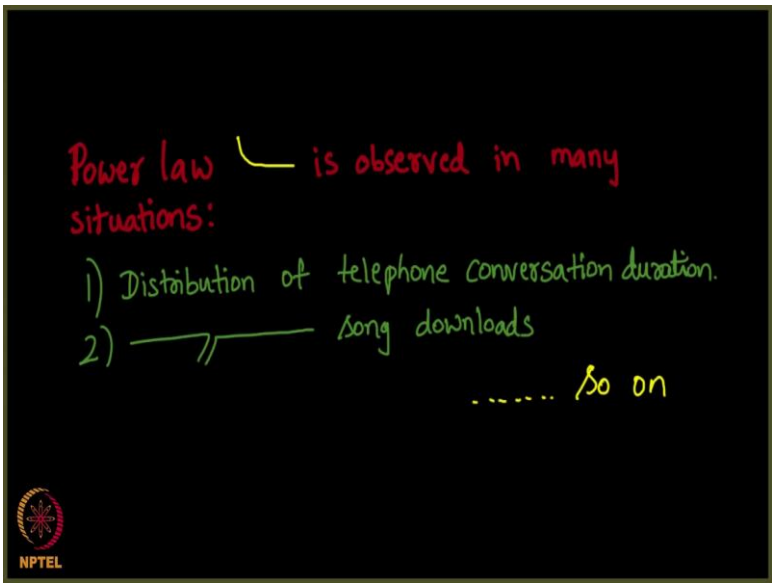
So, it has a very high degrees what we observe remember that. There are many nodes with very high degree it goes on and on. This is called the power law. Power law is basically the drop of the plot the distributional drops ok. And that, that resembles a $1/k^2$ or k^3 or in general $1/k^{\alpha}$. Let us say all right.

So, the power law is a drop, it is observed in many situations, not just in the in. What did we observed in the previous case? We saw the distribution of let us say the incoming nodes of web graph. And we observe it is a power law then, immediate next question you may want to ask is where exactly I see power law apart from these web graphs. In fact, you see them in many situations. You will see them in telephonic conversations let me go back so, that I can show them to you one by one. So, you will see this distribution, this kind of distribution is observed in the telephonic conversation.

For example, what does this means? This means you again plot x and y axis. On x axis you plot the time duration of phone calls and on y axis you plot the number of such phone calls or percentage of such phone calls. Here both are one and the same. And you will observe that it is again I drop like this ok. So, what do I mean by this? How many phone calls have been of 5 minutes duration of whole lot of them? How many of them have been 10-minute duration of whole lot? But relatively less compared to what used to happen for a 5-minute phone call duration. That is a whole lot as you can see 10 minutes is less 15 minutes is a lot less so on and so forth right.

And you do find this extension; you do find there are phone calls which are of a high time duration right. This distribution is also a power law very surprisingly ok. Where else do we see power law? We see that in many places. In fact, this data sets are available online, you can just check.

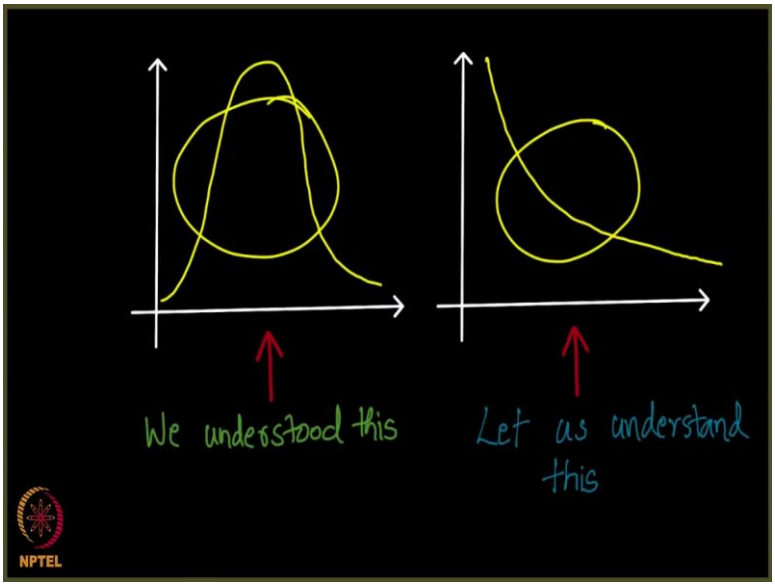
(Refer Slide Time: 09:04)



So, where again do we see the distribution of song downloads right? What are the numbers of songs? What are the songs which are downloaded more than 10000 times from our website? There are some official a song downloads website right.

So, if you go and look at this even these shows a power law ok. This was a exponential drop lines ok. So, which means power law is observed in many places. While we think, it should have been a bell curve in place of bell curve. There is a drop like this ok. So, there are many such networks you can take a look at it. In fact, there are hundreds known till date or may be even more right. I i know at least couple of dozens of a very well-known network that exhibit power law. And people are even looked at why exactly they are exhibit power law which will be the focus of our next lecture.

(Refer Slide Time: 10:02)



So, now we observe that there is a normal distribution. There is a power law distribution right. So, this is this we completely understood right. We understood it completely why was it? Remember I took random variables and I showed you that the sum of random variables is what results in a curve like this right. There is nothing much to explain there and physicists, chemists, biologists you name it people are observed this normal distribution extensively. In statistics this makes the crux of a many observations and they analyse it that is all very nicely neatly and the math of it is completely understood and it is also very nice and elegant. You may have to look at it in case you are interested ok.

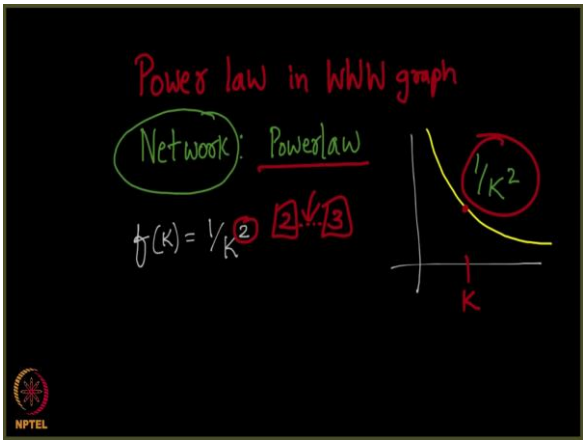
And what we need to understand is we understood this; what we need to understand is this now, right. How does this combined? Let us understand this.

So, our main focus that is coming next is we look at this power law networks which is the web graphs and we will ask this question how did this happen and why did this come here, what exactly made this curve appear like this. While we were actually expecting, what were we expected? You are expecting something like this right; something like this we were expecting, we ended up getting something like this ok. We will see more of it in the next lecture.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Rich Get Richer Phenomenon
Lecture - 118
Detecting the Presence of Powerlaw

(Refer Slide Time: 00:06)



So, we saw in the previous lecture that Powerlaw emerges right. We observe powerlaw in the network of weblets right called the WWW graph the web graph, we observe this we saw this ok. Now, assume you have a network alright. So, before going any further ~~ah~~ I am going to address this question on how to detect power law in given network. Assume you had a network you are given a network and you have this question in mind. The question is in is this network exhibiting powerlaw right. If so, then what exactly is the exponent~~.~~ ~~So~~so, x and y axis and if you can see something like this what exactly is the exponent of this.

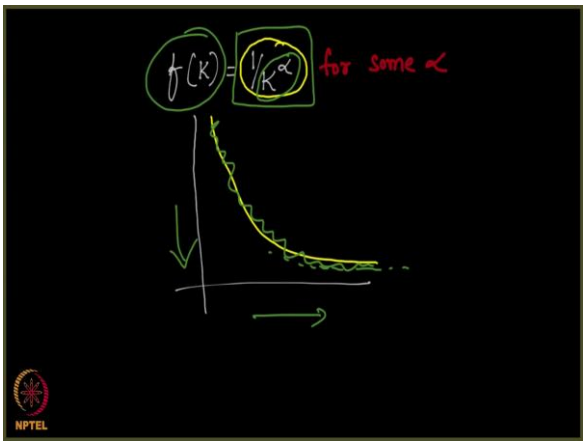
I told you that the exponent in case of the web graph happens to be ~~1 over K square~~$1/K^2$ right. So, how will you find out whether your network that you are looking at exhibits powerlaw or not. So, here is a very easy litmus test that you all can quickly observe and sort of node your head saying it is nothing, but obvious. This is basically as I told you this is your f~~ (of~~ (x) rather ~~f of K~~f(K), if this is K this becomes ~~f of K~~f(K) which is ~~1 over~~

Formatted: Superscript

~~K square~~1/K^2. So, let me note that down and say ~~f of K~~f(K) is equal to ~~1 over K square~~1/K^2.

By the way this exponent here needed necessarily be 2, it is observed to be between 3 and 3, in case of some networks it is 3, in case of some networks it is 2 and so on right. In some networks it is greater than 2 and less than 3, it is somewhere in the middle right in many case, I would like to find out if a network is exhibiting powerlaw or not.

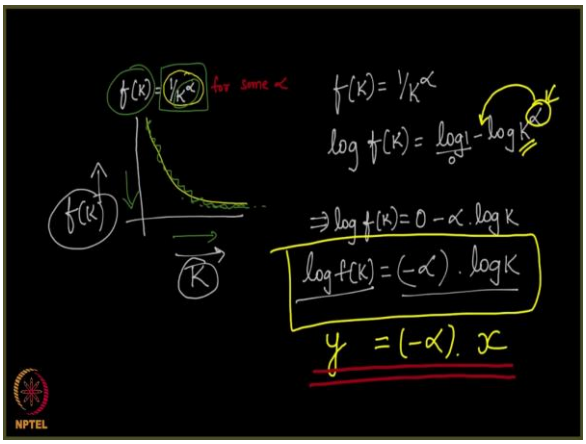
(Refer Slide Time: 02:28)



So, how do I do that? It is it is ~~fairly straight~~straight forward a process. If ~~f of K~~f(K) ~~is equal to~~= 1 over let us say K^{α} ~~to the power of alpha~~ for some α~~alpha~~, I do not know what that α ~~alpha~~ is for some α. ~~alpha l~~Let us say some alpha how do I find this. So, I have a plot given remember you can plot the distribution and you observe that it resembles something like this. And, you know it is something like 1~~/ over~~ K^{α} ~~K to the alpha~~, but you do not know for sure whether it is a reciprocal of a polynomial like this K^{α} ~~K to the alpha~~ or something.

So, what you do is you, what ~~is~~does this curve observe this curve this curve is nothing else, but your ~~f of K~~f(K). And, you ask the question is at of the form 1/K^{α} ~~over K to the alpha a~~As you see as the denominator increases the value sort of decreases that is why it becomes small right ok. How do I see this? I write so, let me make some space from myself here how do I see this?~~.~~

(Refer Slide Time: 03:35)

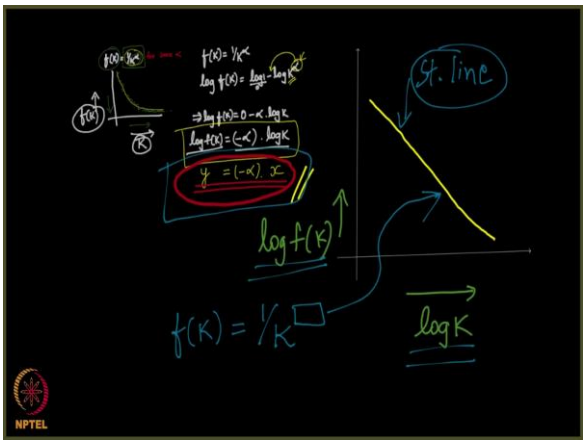


Let me write on this side. So, how do I see this? Given that ~~f of K~~f(K) ~~is equal to~~= 1/-K^α. ~~over K to the alpha w~~What I do is I apply log on both sides very straight forward math, I am sure I am not using anything beyond let us say high school level math which is equal to log 1 ~~minus -~~ log K^α ~~K to the alpha~~. So, basics of logarithms I am sure all of you are familiar this simply becomes log ~~of f of K~~f(K) is equal to we all know log of 1 is 0 0 right.

This becomes 0 ~~minus -~~ log K^α ~~K to the alpha~~ can be brought here that is the property of logarithms, I am sure you all remember minus alpha times log of K so, look at this; look at this. So, what does this mean? Log ~~of f of K~~f(K) ~~is equal to~~= ~~minus -~~ α ~~alpha times~~* log ~~of~~ K, as you know the x axis is your K and y axis was your ~~f of K~~f(K). Now, you see if you were to plot the log of it ok. So, what you will do is you will plot instead of K you will plot log K, instead of ~~f of K~~f(K) you will plot log ~~of f of K~~f(K) ok.

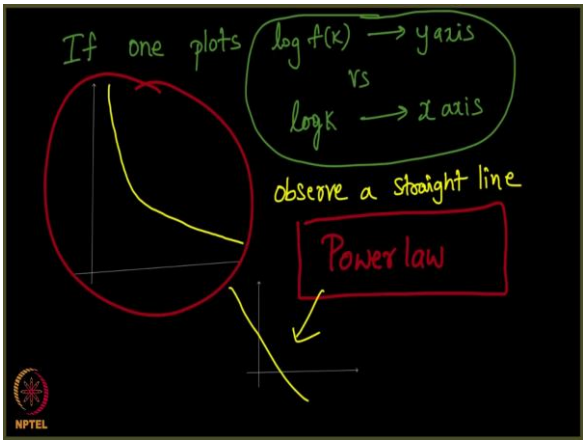
If you observe it, what do you see here? You see that look this is a variable that we call it some y ~~is equal to~~= ~~minus -~~ α~~alpha~~, please note this is a constant minus alpha is a constant because it is the power of K right. K is the variable here right times log ~~of~~ K which I am going to replace by let us say variable x. So, as you can see you are getting equation of a ~~straight line~~straight-line y equals minus alpha times x. So, what does it mean this simply means that when you are trying to plot the log of the graph; the log log of the ~~graph.~~graph?

(Refer Slide Time: 06:02)



Let us say you are not plotting the typical K and ~~f of K~~f(K). You are plotting be clear here I am writing a plot here y axis this is my x axis and if I am trying to plot instead of K, I plot here log K and here I plot log ~~of f of K~~f(K). So, what I will trying to say look at this look at this, I am trying to say that this will ~~actually be~~be a straight line a straight line right. So, actually it should pass through a origin here, but do not worry I am just giving you an example. So, this will be a straight line with slope let us say minus alpha ok, if get a straight line with slope minus alpha.

(Refer Slide Time: 07:43)



So, let me write down a straight line. If you getting a straight line which means that log K versus log f(-K) rather log f(-K) versus log K that you wrote was of this form, which means ~~f of K~~f(K) was indeed of the form 1/~~ over~~ K to the power of something and that something happens to be the slope of this line. So, all in all the litmus test is the following it might sound very technical, but all I am saying is the following let me write that down neatly.

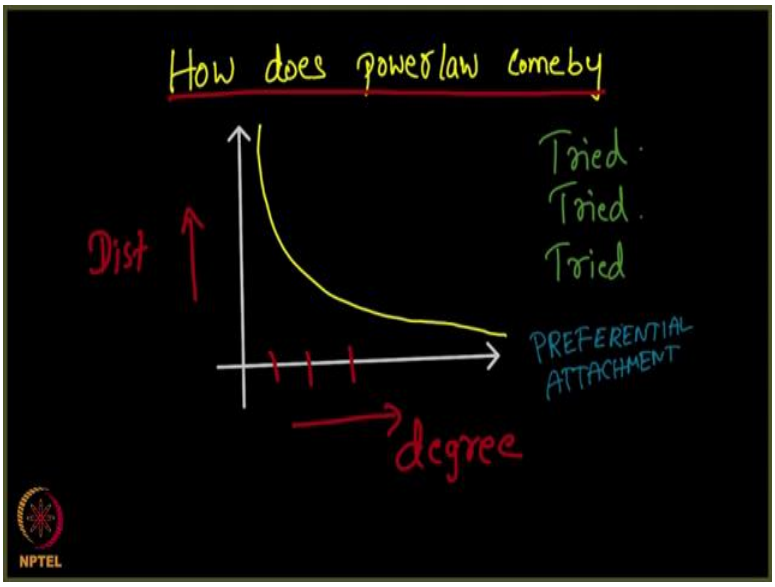
If one plots log of ~~f of K~~f(K) in the y axis; in the y axis versus log K in my x axis, just in case after plotting this if you observe a straight line then you can possibly conclude that there is powerlaw happening here. Now, you might ask me this question when you are given a plot like this let us say like this why ~~cannot I~~I cannot observe this plot and then simply say there is powerlaw happening here. ~~But,~~But then this particular plot can be anything it need not necessarily be 1/-K^{α} ~~over K to the alpha~~ it can be anything right.

So, you never know if it is really of the form 1/~~ divided by~~ a variable to the power of a constant let us say α~~alpha~~. You would not know if it is like this only right. So, what you do is you instead do not take chances when you have when you get a curve like this you try to draw the log log plot; you try to draw the log log plot. This is called a log log plot y axis make it log x axis, make it log values and then try to see you will get a straight line. When you get a straight ~~line~~line, it means that this is powerlaw happening here perfect.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Rich Get Richer Phenomenon
Lecture - 119
Rich Get Richer Phenomenon

(Refer Slide Time: 00:12)

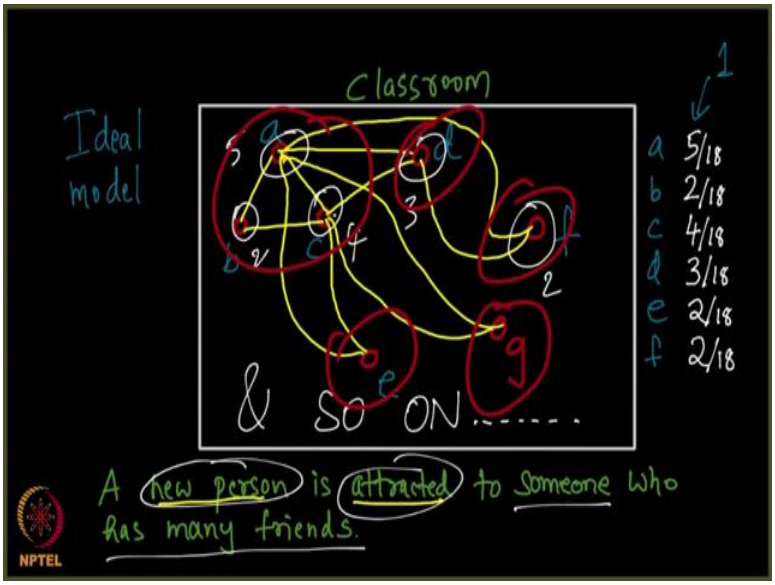


So, let us now make an attempt to answer the big question that we have been asking from quite some time. How does power law come by? What results in this power law ~~right.~~right? So, the interring question just to just for the sake of revision let me repose it. So, when you try to draw the degree let say on the x axis. So, how many vertices are there of degree 1, of degree 2 of, degree 3 if you plot that basically it is the distribution on the y axis you will observe drop like this. The big question is how ~~does this drop~~this drops combine, why does it come in place of something like this we observe something like this why ~~is this even~~this is even happening right.

So, many scientist have tried and tried to understand this, and they have come with their own set of hypothesis and one of the most convincing hypothesis is what is called the preferential attachment is called the preferential attachment hypothesis. It is a hypothesis people are said that there is something called preferential attachment that is happening because of which one observes power law. Now, let us see what exactly ~~is this~~

~~preferential attachment~~this preferential attachment is. So, assume there is a classroom alright.

(Refer Slide Time: 01:59)



So, this is my classroom and there is a person who is sitting here another person another person. So, these 3 people are mutually friends with each other ok. This is a classroom let me write that down, this is a classroom and there only 3 people with only 3 people to begin with let say 1 2 and 3. So, a ~~fourth~~ 4th person comes; a ~~fourth~~ 4th person comes and he tries making friends with 2 of the existing people. So, which means he makes friends with let say 1 and 3. Now, wait a minute observe the degree of different vertices here his degree is 2 right and the degree of this is 3, this degree is 2 the degree of the node 1 is 3 correct. So, let me put this is a oval so, that you know that I am talking about degree here right fine.

Now, when a person comes when a person comes look at the story that I am telling you might question why ~~is this story~~this story is true, for the time being you please assume with the story happens to be true alright. So, this there is this person who come, and he observes that there are 1 2 3 4 there are 4 people here of different degrees correct and he prefers to make friends with. So, let me do one thing I think will remove 1 2 3 4 here for clarity say let me instead call it a b c ok, let me call this a and let me remove 2 here and 3 here and 4 here. Because, it is confusing when I call when I name the nodes as a numbers and then also talk about their degree ok.

Let me call it a b c d and now there is a new fellow e, he looks at these people a b c d and e and looks at their degrees; a has degree 3, b has degree 2, c has degree 3, d has degree 2. And, e must make friends with let say 2 people every person who comes inside makes friends with 2 people, just the way d made friends. He also makes friends with 2 people and who will he choose the story goes like this he will be attracted to people.

So, let me write that down this is a sort of important he will be attracted a new person, a new person is attracted is attracted to someone is attracted to someone who has many who has many friends who has many friends. So, what do I mean by this? It does not means that he is not attracted to people who are less friends. The point I am trying to make is he will be attracted to you know what it is self explanatory has I has I do this small exercise 3 plus 2 is 5 plus 3 is 8 plus 2 is 10.

So, now what I will do is I will write 3 by 10 probability 2 by 10 probability 3 by 10 probability 2 by 10 probability, you will see the sum of all this is actually 1. And, this will be the probability with which e will be attracted to b c d or let say a ok; this is a probability with which you will be attracted. So, e will be attracted to a with probability 3 by 10, b with probability 2 by 10, c with probability 3 by 10, d with probability 2 by 10.

So, what does it even mean? It means that I when I enter classroom, I will be attracted to people proportionately to they with respecting their degree. I will be attracted to c with probability 3 by 10 because the degree is c and then the total number of degree is 10. So, I will be attracted with degree 3 by 10.

So, you probably are confused you are you are not understanding what ~~has this~~this has to do with degree, what has friendship to do with degree. So, the point is simple when you talk to people a person who has a lot of friends will quickly come to the limelight, he is visible to you, he or she is visible to you and you would want to be friends with her more than anyone else, the person who is very visible.

So, the probability of making a new person having a friend is directly proportional to the number of friends that they already have alright. So, let me assume that e actually makes friends ends up making friends with c and in fact, a which means what does it means. I should now update the degree of c it is no more 3 it will be 1 2 3 4 ~~right,~~right; the degree of c is 4 and the degree of a also happens to b 4 degree of a is also 4 ok.

So, this requires me to erase the degree of a here and the degree of c also is to be removed right. And, then the degree of a happens to be 4 and the degree of c happens to be 4 and as you would have guessed I may want to remove all the denominators here. So, it is no more 10 you see what else is it done, it is going to be some of the degrees which is it 12 correct it is going to be 12 here. So, I am going to write 12 12 12 and 12 e also has 2 degree. So, e contributes again 2/ ~~divided by~~ 12 ok. So, let me count the total degree right now e is 2 2 ~~plus~~ + 2 ~~plus~~ + 4 ~~plus~~ + 2 ~~plus~~ + 2 which is oh no, it is not 12 it is 14 you see it is 14. So, let me remove 12 and make it 14 it is not 12 it is 14 good.

So, 14 so, now you are getting what I am doing right. So, 14 for c denominator and d will be again 14, the entire process is actually quite self explanatory, e is also 14. So, entire thing is very self explanatory, why that is because you are just following a simple rule that whenever a new person comes, he becomes attracted to someone who has many friends. It will become very clear one side I writing a new node let say f.

Now, the story becomes slightly complex, but it is easy for you to understand. So, what happens think about it, let me just remove these whatever the degrees here, any way we have a degrees here. So, that is should be enough I am just removing them. So, now tell me where exactly wi~~ll~~l ~~f~~ become friends with, f has now several options a b c d e.

But let us assume f become friends with only 2 people. Now, now you might me why is this restriction to see, it does not look like a ~~real life~~real-life scenario. What do you mean every person has only 2 friends? This is not have it happens in real life, but all I am trying to say is let us for a moment consider an ideal model right. Not all of us will make only 2 friends , but we all know that our friendship budget is sort of the same. We cannot make a lot of ~~friends,~~friends; we cannot make no friends also.

There is some average number and let us have a ideal world here let say each person makes friends with only 2 people ok; f comes here and he needs to makes make friends with 2 people. So, he makes friends with a with probability 4/ ~~by~~ 14, b with probability 2/~~2 by~~ 14, c with probability 4/ ~~by~~ 14, d with 2/ ~~by~~ 14 and e with 2/ ~~by~~ 14 anything can happen. And when he so, this is sort of tossing a coin and then deciding, but you see there is higher probability for f to become friends with a and c right.

So, he problem probably becomes friends with a and with let say d, see I just said these 2 are high probability. Probability does not mean that you always will hit on nodes with

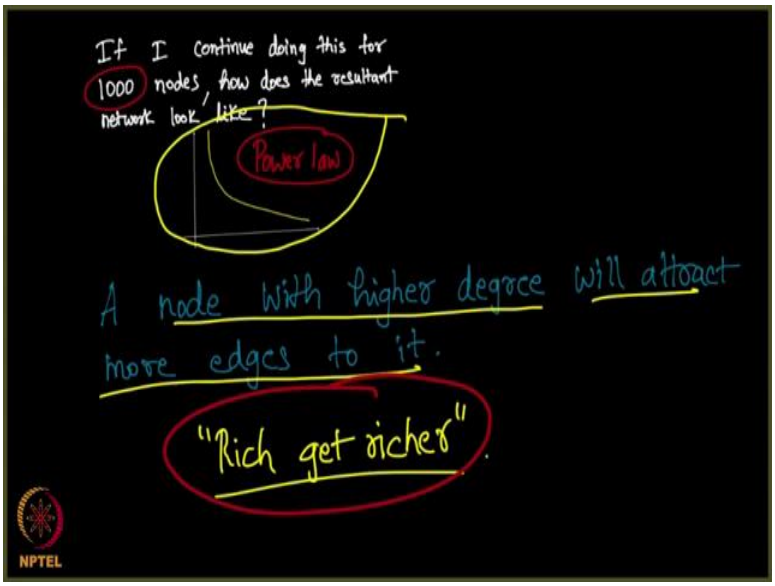
the same with a with a high probability, you might become friends with the anyone here right. You might become friends with anybody here good . So, now let us recompute this probability since see what happens ok. So, here goes my next question why ~~should I~~I should recomputed because, there will be new nodes that is coming namely g ok.

For g to become friends with 2 people g needs to know the revised probability. So, let me remove it and let me again see what ~~is the total degree here 2 plus~~the total degree is here 2 + 3 2 ~~plus~~ + 3 because 1 2 3 4 4 ~~plus~~ + 2 ~~plus~~ + 1 2 3 4 5 for a ok. So, let me write it down 5 for a, 2 for b and 4 for c and 3 for d and 2 for e and 2 for my f.

So, what is the sum? The sum is 5 6 7 10 14 15 16 17 18 is the sum. So, the denominator will become 18 right now ok. I hope you are getting a hang of it right now, as I am doing this repeatedly. I am spending sometimes doing this manually. So, that you are understand what happening and later on we are going to write a piece of code and show you what exactly is happening here.

So, now g has come g has come, g must become friends with couple of people and he will respect this probabilities and become friends with 2 people. And, as you can see there are there is roughly one thirds probability that he will be friends with a right. And, good properly that he will be friends with c as well correct and also yeah 3 1 18 is 1 norm 6 right with d as well. So, g ends up becoming friends with let us say a and let say c and so on and so on dot ~~dot~~ dot ok. As I keep doing this the question that I would like to ask you all is the following, let me write that doubts that it seems into your mind.

(Refer Slide Time: 13:47)



If I continue doing this process this for 1000 nodes let say, starting from the first 3 nodes as you know; let me go to the previous slide we start with this first 3 nodes. And, we went on adding more and more and more nodes correct right. As I keep doing this as I keep doing this if I keep doing this for 1000 nodes how does the graph look like, how does the resultant network look like.

So, what is happening let me go back and see what is happening. A new node always makes a 2 friends right, a new node makes friends with 2 people always and the next node that comes make friends with the existing people respecting the degree right. Any node that is coming becomes friends with the existing node by respecting the existing degree correct ok.

So, look like how does it look like? It looks like you will be started to observe that it looks like a network, where the degree distribution indeed follows powerlaw. Now that is the climax of a discussion, it indeed follows powerlaw. Such a network which grows by respecting the degrees of the existing people and then grow slowly node by node it follows powerlaw ok.

And, let us go back and then see what happened here, you see people nodes who already had higher probability had the propensity to attract new nodes right. It this continuous for 1000 nodes, you will observe that higher degree nodes will become more and more

rather it will become the degree will become higher and higher for nodes the whose degrees is already higher ok.

I repeat that I repeat that is that it is clear to you ok. So, let me is write that here a node with higher degree higher degree will attract more edges to it, why is that why is that isn't that obvious that is what we have been discussed. Because, a new node that comes that is comes will become friends with the existing a node that comes will become friends to the existing node that already has higher degree. Because, that is are that is the protocol that is the algorithm right. So, he will have many other options, but he will choose that node with higher degree already correct; that is what we discussed here right correct. So, a node with higher degree will attract more nodes to it.

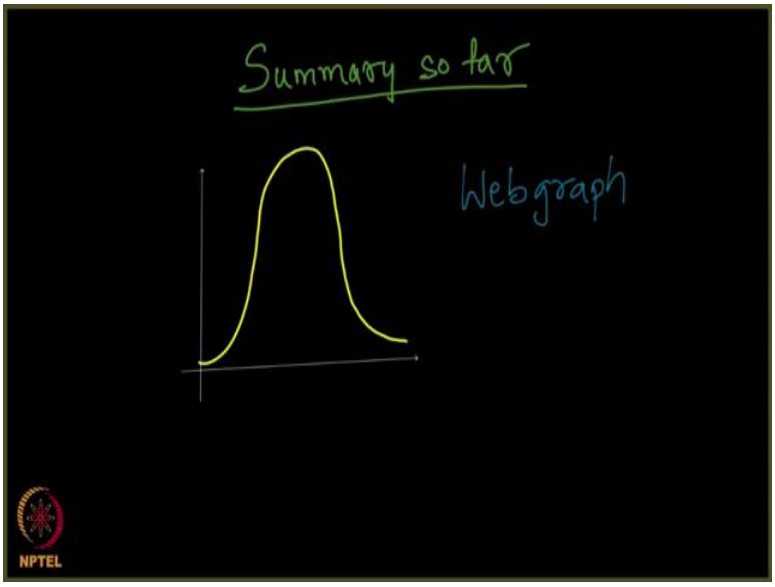
Now, this process this process is called rich get richer process; I am sure it makes sense the very terminology make sense to you. What you mean by rich get richer? People who already have high degree are getting more and more degree because, that is what algorithm does you see. And surprisingly such an algorithms such of protocols such of mechanism of developing networks results in the emergence of powerlaw. It also a node with higher degree will attract more adjust to it. This phenomena is called the rich get richer phenomena and we observe powerlaw in such networks.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Richer Get Richer Phenomenon

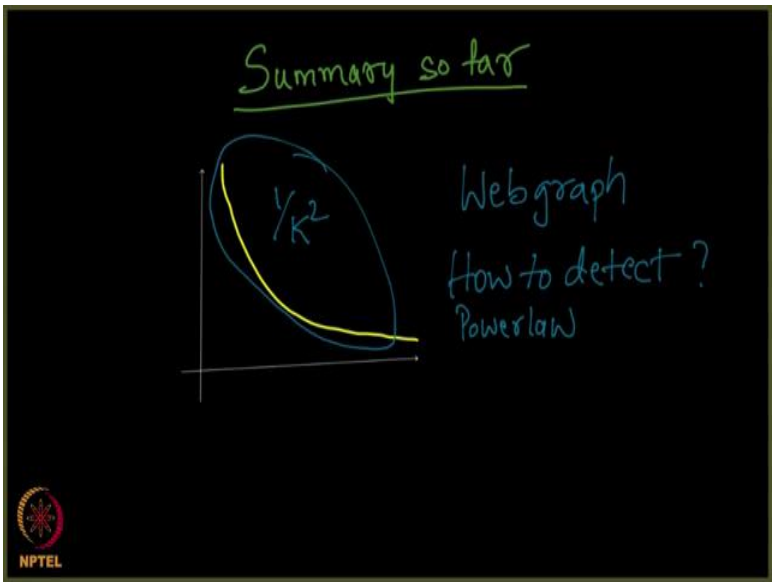
Lecture – 120
Summary So Far

(Refer Slide Time: 00:07)



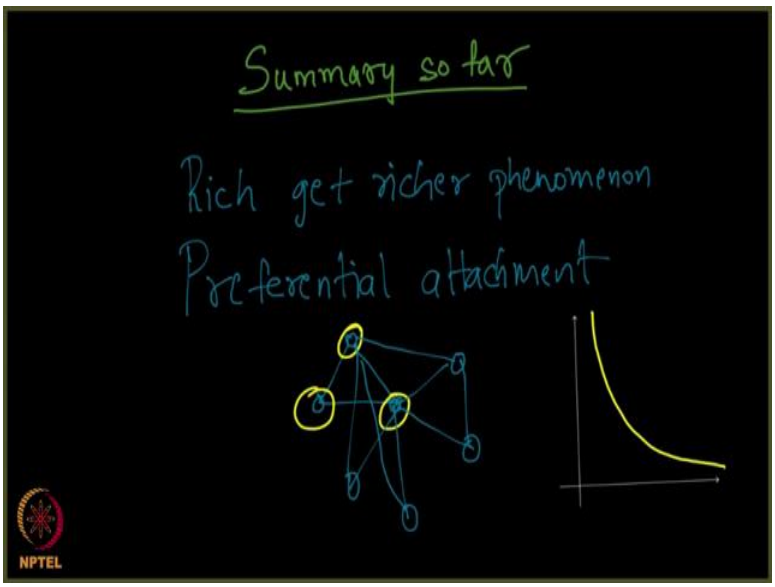
So, what are we seen so far, let us summarize it. The summary so, far is the following: we observed the emergence of what is called the normal distribution right. We saw that it is always a curve called the bell curve, the height example of a town right. In a town if you take the heights of different people you will observe this, but then we saw that it is in fact, not the case if you were to be seeing what is called the web graph or the (Refer Time: 00:42) graph, www graph right.

(Refer Slide Time: 00:49)



What do you observe? We I told you that we observe a drop like this and even I told you that the drop is basically a function $1/K^2$ right. And, we even saw how to detect such a thing let us such a thing happening in a network, how did detected right. How you detected power law that was the question we saw, next right perfect and then we switched gears and we saw the process of rich getting richer right.

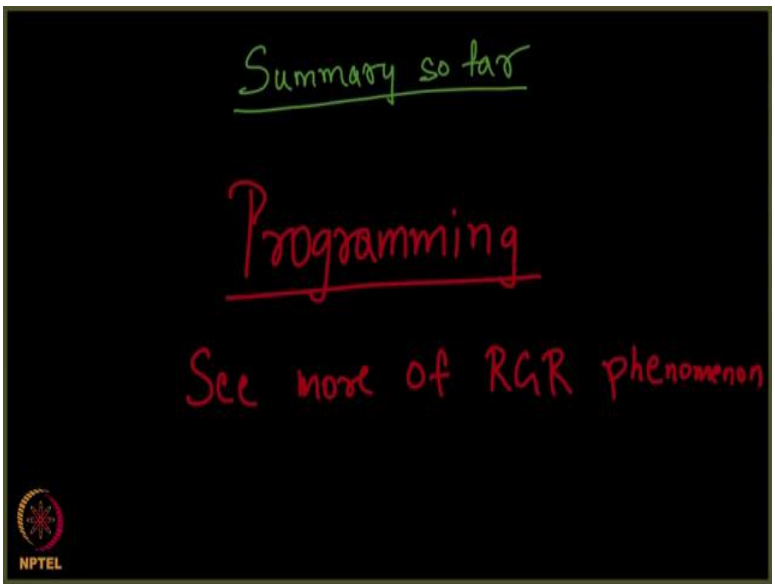
(Refer Slide Time: 01:24)



Rich get richer phenomenon and what was that, if you can recollect it was what is called the preferential attachment, preferential attachment. What we say? We said that given

node we start with let say 3 nodes, just as an example we start with 3 nodes and then and a new nodes comes. It respects the degree of the existing nodes and based on the degree proportionately it becomes friends with a couple of people alright. And, as we continue this process as we continue this process, we observe that when we plot the degree distribution of such a graph we observe the power law alright. There is a summary so for and now what we will do is you will switch gears and we will try looking at some programming.

(Refer Slide Time: 02:31)



We will try programming whatever we have learnt so far and try to see if we are able to make the same observations that I have hypothesize so far. So, after programming we will get back and then see more of rich getting richer phenomenon. So, now let switch to Python and then see some really cool programming observations.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

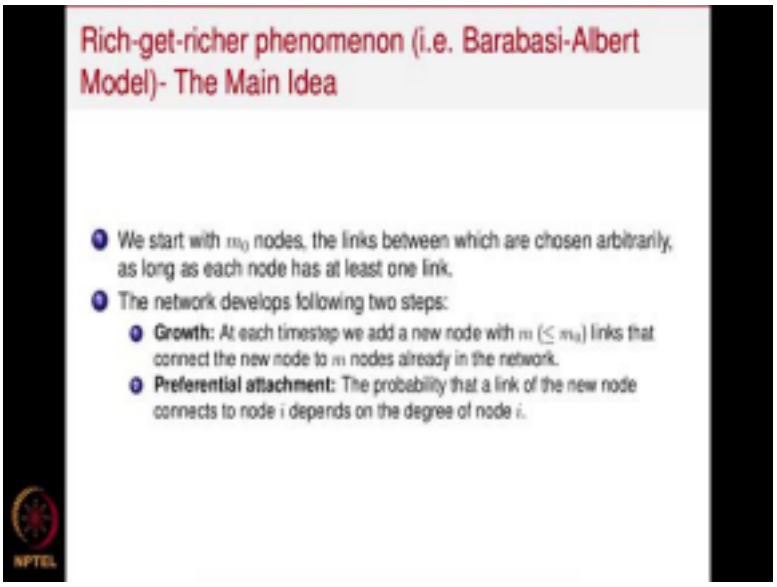
Rich Get Richer Phenomenon
Lecture - 121
Implementing Rich-getting-richer Phenomenon
(Barabasi-Albert Model)-1

(Refer Slide Time: 00:05)



Hey everyone, in this video sequence we are going to implement an interesting phenomenon that is called Rich getting richer Phenomenon. It is also called Matthew effect and it is also called Preferential Attachment. Now, there is a particular model that is called Barabasi Albert model which is precisely based on this method that is called preferential attachment. So, we are going to implement Barabasi Albert model in these subsequent videos and that be same as implementing rich getting richer phenomena as well. Before we get into the steps of implementation, let us look at the main idea behind this concept.

(Refer Slide Time: 00:40)

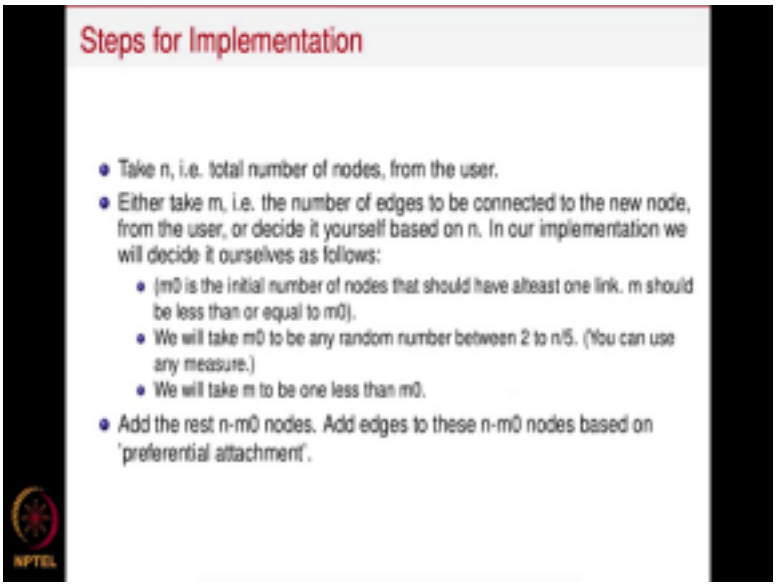


So, the main idea is that we are going to start with some m0 number of nodes. And, these nodes are going to have some edges across each other. The only condition is that every node should have at least one edge; this means that we start with an existing network of m0 nodes. And, the new node and the new notes get attached to this existing network one by one. Now, how do they get attached? There are two properties that every node follows, and the first thing is that every node gets attached to some m number of existing nodes. So, for example, say m0 is equal to say 5 and m has to be less than equal to m0.

So, assume m0 is equal to 5 and m is equal to say 3. So, we will start with an existing network of 5 nodes and 1 new node will come and it will get attached to some 3 existing nodes. And, then second node will come that will also get attached to these existing 3 nodes and so on so, this keeps happening. Now, how do we decide which 3 existing nodes should get attached to the new node, that is decided by a preferential attachment. Now, what is preferential attachment? This means that the more connected a node is, the more likely it is to receive links with a new node.

In other words, more degree an existing node has more probability it will have of connecting to the new node. So, this is how we will keep adding new nodes to the existing network. This was the main idea behind rich getting richer phenomenon. Now, let us look at this steps that we are going to follow for the implementation.

(Refer Slide Time: 02:26)



So, I am going to required value of n, n that is the total number of nodes in the network. We will take the value of n from the user, the other thing that we need is the value of m that is equal to the number of edges that every new node is going to get attached to; basically the number of edges that every node will introduced into the network. So, we will take value of m from the user or we can have our code decide the value of m automatically.

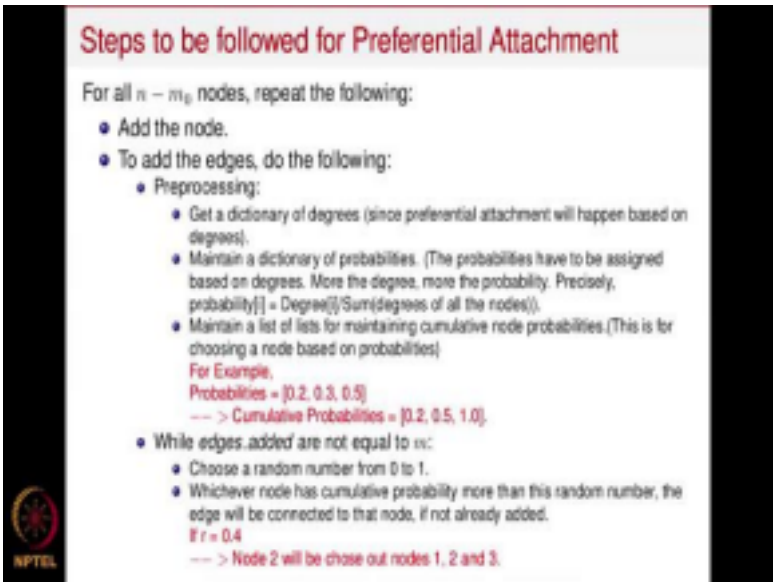
In our code we will decide the value of m based on n, it is up to you can do whatever you feel like ok. The other thing that we need is m0, as I told you m0 is the number of nodes in the existing network, in the initial network. The only condition is that m <= m0. So, we need n, we need m and we need m0; these 3 values we need, and the only condition is that m should be less that equal to m0.

So, in our code we are going to have m = m0 - 1, you can just fix it on any a new value that you wish to have. So, in our code we will have m0 to be any random number between 2 to n/5, this is this is just an assumption we are taking; you can go ahead with any value of m0. You can fix on a value m0 = 4. So, every time you will start with an initial network of 4 nodes and m, we will take be m0 - 1 you can take any value.

Now, we have a network of m0 nodes, the next step is to keep adding new nodes to this existing network. So, the number of nodes that are going to be attached is equal to n - m0 because, n is the total number of nodes. So, these n minus m0 nodes are going to be

attached to existing network based on preferential attachment as I explained you. Now, let us see how we are going to implement preferential attachment ok.

(Refer Slide Time: 04:38)



So, we have to add n - m0 nodes one by one. So firstly, we will add the node to the network. The next step is to add m edges to this node to the existing nodes in the network. Now, how are we going to decide the nodes that are going to be attached to this existing node? As I told you we need we need the probabilities and the probabilities are going to be decided based on the degrees that the nodes have, more the degree more the probability right. So, for that pattern we need to keep a track of the degrees of the nodes. So, we will do some pre processing for this keeping track. We will maintain a dictionary of degrees for every node because, we the degrees basically going to decide the probabilities.

Second thing is we will maintain a dictionary of probabilities. So, probability of a node getting attached to a new node will be equal to the degree of that node divided by the sum of the degrees of all the nodes right. So, more the degree more the probability so, we are going to maintain a track of the probabilities in a dictionary. Now, apart from that we are going to maintain a third thing, we will basically maintain a list of list because we want to keep an order of the things and order of nodes dictionary does not let you maintain an order. So, we are preferring to keep a list of lists for maintaining cumulative node probabilities ok.

Now, let me tell you why we need that. So, whenever you have to choose a node based on a probability this is a standard technique that we use, although you can device some other method is well. We are going to follow this method of making use of cumulative probabilities. Now, let me explain you what that is using an example. Assume there are 3 nodes and they have the properties 0.2 0.3 and 0.5, we are going to maintain a list that is cumulative probabilities.

So, the first node we have cumulative probability 0.2, the second we will have 0.2 plus 0.3 that is 0.5, the third will have 0.5 plus 0.5 that is 1.0. So, this is the less that we are going to maintain, I will show you how we are going to use this list. So, when new node has come, we have to connect this new node to existing m nodes for that we will choose random number between 0 and 1 ok. And, then whichever node has cumulative probability more than this random number is the node which is going to be connected to the new node.

Let me show that one example. So, assume r comes out to be 0.5 0.4 let us look at the cumulative probabilities list that we just created. So, first nodes cumulative probabilities 0.2 which is less than 0.4; so, we will go ahead we will check the second one, 0.5 is the second value, 0.5 is more than 0.4. So, we will stop there since r is less than or equal to 0.5 the second node is basically the node which is going to get connected to the new node. You would have understood this method, but do you understand the main idea behind it.

So, the main idea is that more probability node has more likely it will get connected to the node, this is what we have to implement. Now, try to understand how this particular method is helping us implement the same thing. So, basically more probability a node has more window it will have in the cumulative probabilities list. Now, what do I mean by that? So, as you can see just take a look at the cumulative probabilities list. So, we have first nodes cumulative probability to be 0.2. So, it is window will be 0 to 0.2, the window for the second node is 0.2 to 0.5 ok, the window for the third node is 0.5 to 1.0 ok.

So, now you can see that the window for the third node is more than the window for the second node and the window for the second node is more than the window for the first

node. So, wider a window is more likely it is that the random number is going to fall into that window ok. So, assume a node has probability 0.1, it is window will be 0.1 big, if a node has probability 0.6 it is window will be 0.6 wide and more likely it is that the random number is going to fall into that window ok.

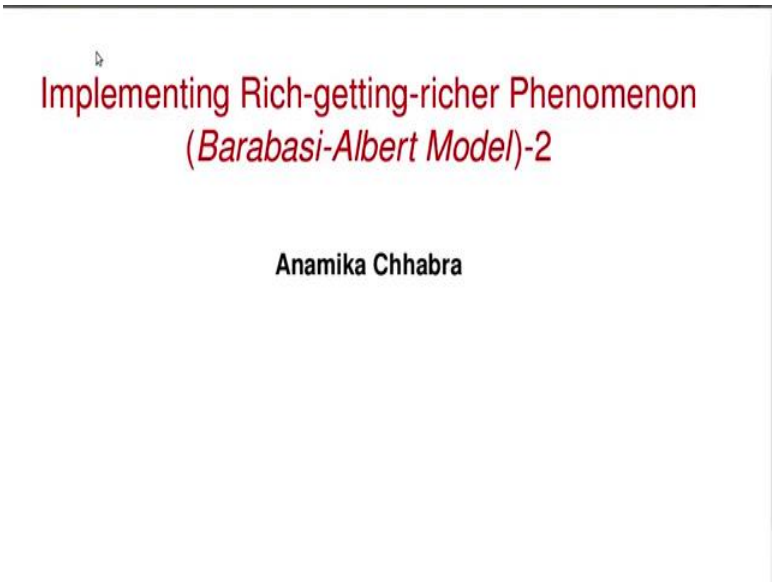
And this precisely we are implementing here. So, if a node has more probability the random number is going to fall into that window more likely and that is how we will connect that node to the new node. So, that is the main idea. This is how we are going to add the edges to the new node, and this is how we are going to add new nodes up to the point that we get total n nodes in the network. So, this is the main idea.

In fact, not the main idea this is detailed steps of implementation that I have discussed with you. I would suggest you since I have gone to detail into the steps of implementation, it will be nice if you just start of yourself, coding yourself and implement yourself and do not follow the subsequent videos line by line. And, just compare the results with the results of the subsequent videos that will be a nice practice. So, let us get started with the coding part.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

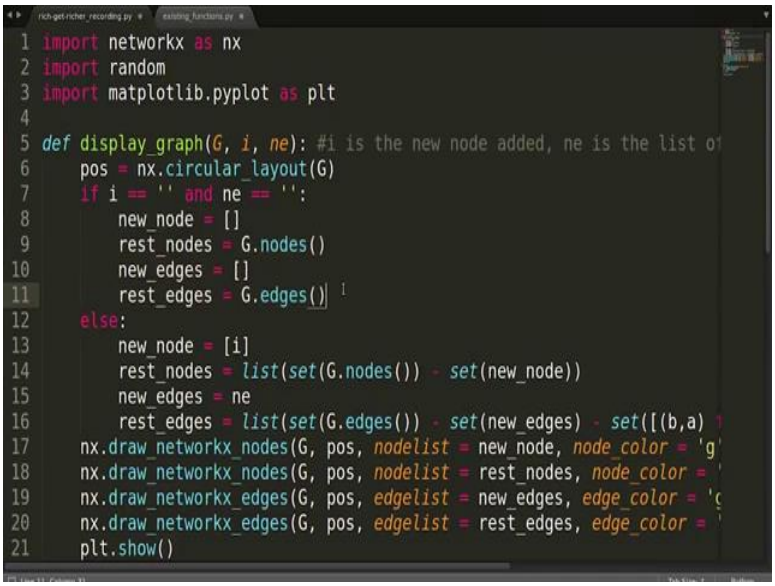
Rich Get Richer Phenomenon
Lecture – 122
Implementing Rich-getting-richer Phenomenon (Barabasi-Albert Model)-2

(Refer Slide Time: 00:05)



In the previous video, we discuss the main idea behind Rich-getting-richer Phenomena. And we also discuss this steps that we are going to follow for implementation. In this video we are going to start off with implementation.

(Refer Slide Time: 00:19)



So, let me import some packages that we are going to need, we are going to need the network. We are going to take random number, so we will meet random package. We are also going to display the networks, so let us import matplotlib ok.

So, let us start our main function ok. So, if you remember from the previous video, if you remember the outline if you have not seen the video, you may go back and see the steps at we are going to follow here. So, we require the values of n, m and m0, n is the total number of users, m is the number of edges that every new node will get attached to, m0 is the number of nodes in the initial network that we will start with.

Let us take the value of n from the user. So, we can use the function raw input and in case you are using Python 3.x you can use input function instead of raw input. So, let us use this function, raw input and as the user entered the value of n all right. Next, we need the value of m0 as I told you can initialize m0 to be some 3, 4, 5 whatever you want. Let me take m0 based on n that the user will pass. So, I plan to take a random value between 2 is that is the minimum number of nodes that should be there in the initial network and n divided by 5; to n divided by 5 I will choose the random number between 2 to n/5. So, assume n the user entered n to be 100 then I will take a random value between 2 and 20 that is how I will start the initial network.

In fact, if the user enters a very high value of n that this might not be a good idea, you can it is it is actually better to fix up a value of n0 to be 3, 4 or something. For example,

user enters n to be 1000 then your initial networks will be between a random number between 2 and 200 right. It is fine you can take any value of m0. And if you remember the only thing is that m should be less than equal to m0. So, here we will take m to be m0 – 1. m = m0 - 1. And we have to start with an initial network with m0 nodes.

So, as of now we have not to means started the graph. So, let us start the initial graph with m0 nodes. And the condition is that every node should have at least one edge. So, I will have a path graph because, there every node has at least one edge you can do any other thing like, you can add the edges one by one just making sure that an every node has one link at least. You can manually add the edges, so it is up to you. I will take a path graph, I will write nx.path graph of m0 nodes ok.

Now, to start off I wish to see how the graph looks like. So, that we can see the changes that that keep happening to the graph at every time stamp, I am going to make use of a function display graph that we created in one of the previous videos. So, I am going to just take the function from there and I will briefly explain it here, I will not write the function line by line here.

(Refer Slide Time: 04:37)

```
def display_graph(G, i, ne): #i is the new node added, ne is the list of
    pos = nx.circular_layout(G)
    if i == '' and ne == '':
        new_node = []
        rest_nodes = G.nodes()
        new_edges = []
        rest_edges = G.edges()
    else:
        new_node = [i]
        rest_nodes = list(set(G.nodes()) - set(new_node))
        new_edges = ne
        rest_edges = list(set(G.edges()) - set(new_edges) - set([(b,a)
    nx.draw_networkx_nodes(G, pos, nodelist = new_node, node_color = 'g
    nx.draw_networkx_nodes(G, pos, nodelist = rest_nodes, node_color =
    nx.draw_networkx_edges(G, pos, edgelist = new_edges, edge_color = 'g
    nx.draw_networkx_edges(G, pos, edgelist = rest_edges, edge_color =
    plt.show()
```

So, let me call that function display graph before that let me copy it. So, this is the function that we created one of the previous videos, I am copying it and pasting it here, I brief out for the once you are not see in the previous videos.

So, what this function is doing? We are passing the graph G that has to be displayed. We are passing i, i is the new node to be added and ne is the list of new edges that are going to be added. So, basically at every iteration I plan to call this function display graph and I also want to display the new edge that got added in this iteration, what did I say; I want to display the new node that got added, I and I also want to display the new edge is that got added. So, I am going to change the colours of this nodes and edges. So, that its nice to see how the nodes are getting added for that only I have created this customized function.

I am going to use a circular layout ok. So, I will use this function nx.circular_layout(G). And since, I am going since i need to display the colours of the new node and new edges differently. So, what I have to do here is that. Firstly, I am putting this condition ok. So, initially there will be no new node no new edge. So, i and ne will be empty that is what will happened at the first before any iteration takes place initially right. So, new node will be. So, we will maintain a list of new nodes and we will maintain a list of new edges. Initially, they will be empty and rest of the nodes which is all the nodes in the graph, rest of the edges which is all the edges in the graph. So, we require these 4 list previous nodes, new node previous edges, new edges, these 4 list, we need ok..

(Refer Slide Time: 06:43)

```
22
23
24 def add_nodes_barabasi(G, n, m0):
25     m = m0-1
26
27     for i in range(m0+1, n+1):
28         G.add_node(i)
29         #add the edges one by one
30         #preprocessing
31         degrees = nx.degree(G)
32         node_probabilities = {}
33
34         for each in G.nodes():
35             node_probabilities[each] = (float)(degrees[each])/sum(degree
36
37         node_probabilities_cum = []
38         prev= 0
39
40         for n,p in node_probabilities.items():
41             temp = [n, prev+p]
42             node_probabilities_cum.append(temp)
```

In case i and ne are not are not empty, there are some values in them, what are we going to do is, new node list is going to have i new edges list is going to have ne, which we

passed as a parameter. Rest of the edges will be all the edges, I am sorry rest of the nodes will be all the nodes minus the new node, rest of the edges will be all the edges minus new edges minus the intersection right; minus the common edges basically, let me explain.

(Refer Slide Time: 07:17)

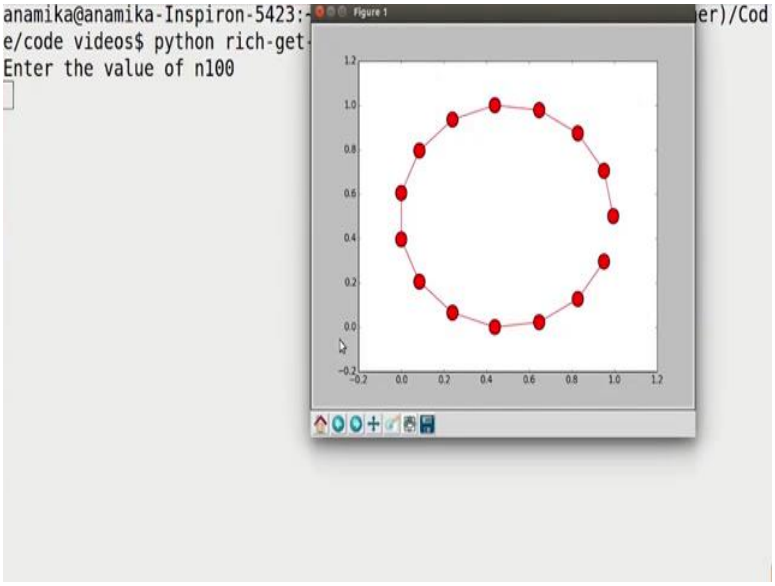


So, it should take b a and a b to be the same edges right. So, that is why we are deducting that those edges ok. So, then we are displaying. So, since you want the graph to be customized having different nodes having different colours different edges different having different colours, you will have to make use of this function draw networkx nodes and draw networkx edges. So, separately you are calling them.

So, first function will be to display the existing nodes, second function will be to display the new nodes or vice versa. First function will be to display the new edges and second function is to display the existing edges ok. So, look at the parameters quickly we will pass the graph G and then the position that we already created and the node list that has to be given this particular colour. And we are specifying the node colour to be green here and the existing nodes, we are displaying red, it is new edges, we are display in green and you can also change the style and displaying the new edges to be dotted so that, we can precisely differentiate. And we are existing edges, we are displaying that. So, this was about the displaying part, now we are calling this function display graph and ok.

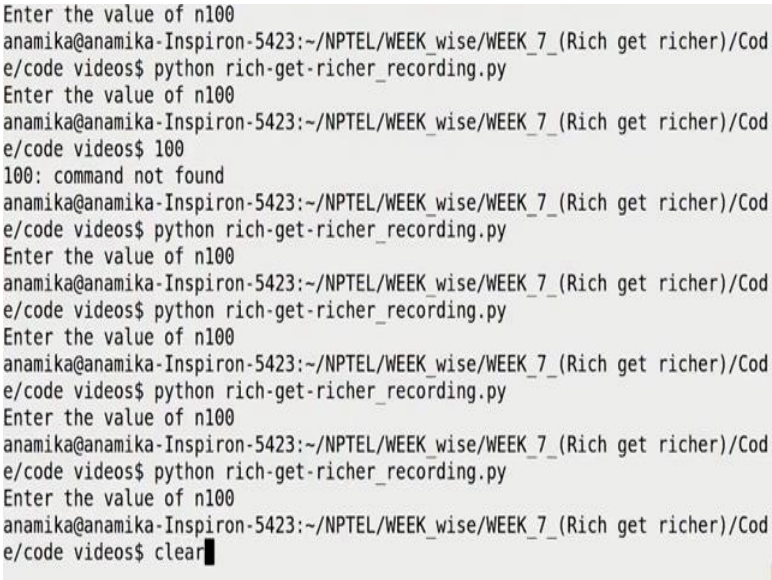
So, we are calling this function, we need 3 parameters G i and ne. So, let us pass G here and i initially, there is nothing, there is no node and the edges also will be empty ok. So, let us call this function let us call main to check the functioning of our existing code ok.

(Refer Slide Time: 09:33)



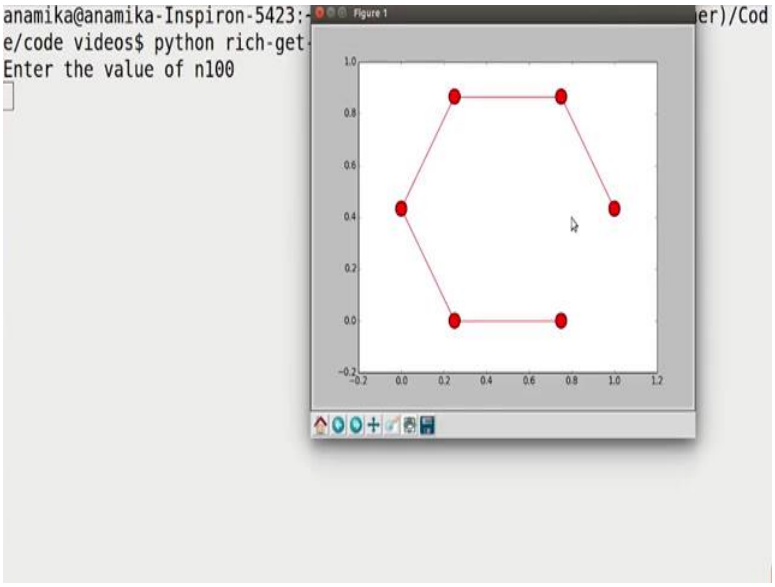
Let us call this file, enter the value of n, I am going to get 100 ok. So, this is the initial graph this is the initial graph as you can see, we had chosen m0 to be random. So, this is the number of nodes that the code has chosen. This is the initial graph and we are going to add the nodes one by one to this graph.

(Refer Slide Time: 09:56)



I can execute it again to see and this is taking these many nodes initially, also taking quite good number of nodes initially, I am sorry; I am sorry, second good number of nodes by chance ok.

(Refer Slide Time: 10:27)



I am just trying to see whether it case. So now, it took 1, 2, 3, 4, 5, 6 nodes initially.

(Refer Slide Time: 10:32)



So, it always keeps changing you can fix it or you can have the good design. So, this is the initial graph and we are going to add the nodes to this graph.

Now for adding the nodes let us create a function let us create a function add nodes since, we are going to follow Barabasi Albert model, let me name that accordingly Barabasi ok. We should pass the graph there we should pass the value of n the total number of nodes, we should pass the value of m that is the number of edges and it should written the graph that is what we are going to we will be displaying ok.

So, we have to create this function, now you see there are m0 nodes into the existing graph already ok. So, we need the value of m0 here right because m0 is the number of nodes already. So, I should better pass m0 over here and m0 over here and let me decide the value of m in the function itself. So, I am removing this and I am passing G, n and m0. Let me decide the value of m here, I will write m is equal to m0 minus 1 ok. So, we have m0 nodes and we have to add n minus m0 nodes more.

So, I will start a loop for every node for i in range m0 + 1 is the next node that is going to be added and we have to stop at n. So, since it is a range function, I will put n plus 1 because, it excludes last value. So, this is a loop to add nodes one by one. So, we will add the first node G.add_node, the node is going to be I now we are added the nodes now, we have to add the edges. If you remember from the previous video, we have to do some pre processing. So, let me see that and under pre processing before that let me also write add the edges one by one ok.

Now, what is the pre processing? We need to choose the nodes based on the probabilities, which are based on the degrees. So, we need to maintain the degrees. So, let me take dictionary degrees = nx.degree(G) this written as a dictionary of dictionary, where the key is the node and the value is the degree ok. Now we also need to maintain the probabilities, which are which will be based on the degrees. So, let me create dictionary and node probabilities inside probabilities no probabilities is equal to empty dictionary this is what we are going to create, now how do we create it?

As I told you the probabilities probability of node i is equal to the degree of i divided by the sum of the degree of all the nodes. So, we should get a start loop here to assign the values to this dictionary. So, I will write for each in G.nodes node probabilities for each is going to be degree the I am sorry the degrees are there in these dictionary degrees of each, we will give us the degree of that node and that should be divided by the sum of all the degrees. Now where are all the degrees? degrees.values, if you write degrees or

values you get a list of all the degrees, we are going to sum them up and since we are dividing let me do the (Refer Time: 14:59) casting float ok, we should work.

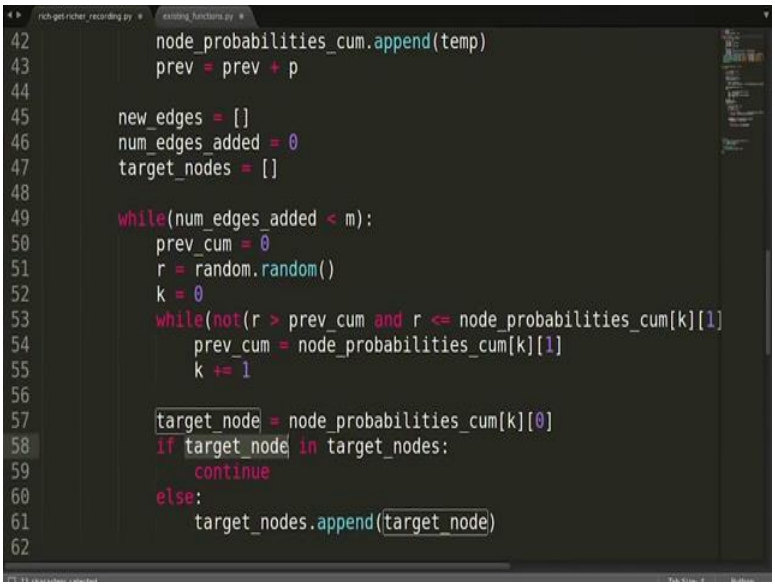
So, we have now created the dictionary node probabilities ok if you remember from the previous video. Now, the next step is to create list that is cumulative node probabilities since, we want then, to be ordered I got create list of list and not the dictionary. Let me show, you how will do that node probabilities cumulative is equal to a list ok. So, we have to take the probabilities from these dictionary node probabilities one by one and based on that we will compute the cumulative probabilities. So, at this point let me start loop, I will write for n comma p in node probabilities.items.

Now, what do we get here? Node probabilities is a dictionary.items will give us a list of topples and topple we will have 2 things n , p that is the node comma the probability. So, we are caption that in n, p here and we have to store the cumulative values in the in the list of lists that is node probabilities cumulative here. So, let me initialize the probability to be initial probability to be 0 you will understand as I go on, I am plating a list temporary list, let me it temp that we will appending that we will appending to this list ok..

So initially, this list has to have node comma the cumulative probability. So, the cumulative probability for the first node is going to be basically, 0 plus it is own probability, if you remember from the previous example, we took in the previous video. So, the cumulative probability for the first node is going to be it is probability itself. So, we have going to add 0 it, and the cumulative probability for the second node is going to be the cumulative probability for the previous node plus the probability for the current node. So, we need to maintain this variable previous.

Now, this is going to be the list for the first node and we will append it to the node probabilities cumulative.append(temp) right..

(Refer Slide Time: 17:44)



And the previous should now be updated. So, I will write previous is equal to previous plus plus p, because p was the probability for the current node that p appended. In order to add the edges, we will now create another loop that we make use of this cumulative probabilities list. One thing that we have to take care of is an edge that has already been added should not be added again ok.

So, we need to keep a track of the number of edges added that is one thing, second thing is we also have to keep track of the edges that I have been added. So, we need to maintain a list of the edges that we have added for a given node so, that we do not repeat the edges. So, in case and if you do not keep a track of that that is if you do not keep a track of the edges that we have already added, we might end up adding the same edge again. So, in that case it might happen that some nodes are adding lesser edges than n ok.

So, let me create a list new edges is empty; here, we will store the new edges and number of edges added number of edges added for this particular node right initialise to 0. And in order to check whether the same existing node is not going to be connected again, I am going to create list target nodes initialise to 0, I will show you how these list are going to be add going to be used.

Now, we have to add the m edges to the new node. So, I will start loop here why? Number of edges added is less than m what do we do? We have to chose an existing node based on it is probability for that we need to random number. So, I will take r is

equal to random.uniform, which is going to give me a random value between 0 and 1 you can also use random node random, you can use that this one I think just check, what is the difference between these 2 random.random gives value between 0 and 1 right ok.

So now so, if you remember the only difference between random and uniform is that in random you do not need to pass any parameter and it gives you a float value between 0 and 1 and uniform. You can pass the parameters a comma b where the floating point value that the function will return will be between a and b. So, if you want a random floating point between 0 and 1, you can use uniform bit parameter 0 and 1 that was about the probabilistic of these functions.

Now, we have to choose an existing node based on this r. So, if you remember there is going be your window which is cumulative probability list will provide us. So, we have to check the first nodes cumulative probability, if r is less than equal to cumulative probability of the first node, we will stop there otherwise; we will go to the next node, if r is less than equal to the cumulative probability of the second node we will consider that else we will go on. So, let me start loop here why not r is greater than we need this starting point as well. So, let me let me take a variable here previous cumulative is equal to 0. So, this is just create the windows we need to keep track of the previous nodes cumulative probability as well in order to find the window.

So, I will write while r is greater than previous cumulative, and r is less than equal to node probabilities cumulative of the first node ok. So, we need keep a track of the first as well k is equal to 0, first node k. And if you remember node probabilities cumulative is a list of list, where every member list has first element as the node and the second element as the cumulative probability. So, we will write I am sorry one here right. So, while this does not happen that r is not into that window, we will keep going on with the next element.

So, in our process previous cumulative is going to be updated with the cumulative probability of the current node and k will be implemented ok. Now whenever it finds a node, whose cumulative probability is more than this random number it will come out of the loop and when it when it comes out of the loop, it has found the node that is the target node. So, target node we are taking a variable target node, sorry target node is equal to now node cumulative probabilities is a list of lists and every list has first

element as a node and second implement as cumulative probability. Now we need the element here the node here. So, we will write k and here 0. So, we will get the first element that is the node that is the node.

Now, we have to check whether this node has already been connected to the new node or not? So, for that wait there is a spelling mistake target write e. So, we have already maintain this target nodes list, which will contain the all the target nodes. So, if though new node which we have chosen is already in target nodes, we will continue finding another target node if the if the target node is not there in this list, we will added there ok. So, we will write it is target node in this list target nodes, what will do? We will continue ok; we will find another target node otherwise this target node is not there in target nodes list. So, we will appended there target nodes.append this new node target node.

So basically, if the node that we have chosen is already in the list of nodes that is the target nodes. We will continue means we will go back here, and m is not implemented.

(Refer Slide Time: 25:24)

```
53              r = random.random()
54              k = 0
55              while(not(r > prev_cum and r <= node_probabilities_cum[k][1]
56                  prev_cum = node_probabilities_cum[k][1]
57                  k += 1
58
59              target_node = node_probabilities_cum[k][0]
60              if target_node in target_nodes:
61                  continue
62              else:
63                  target_nodes.append(target_node)
64              G.add_edge(i, target_node)
65              num_edges_added += 1
66              new_edges.append((i, target_node))
67
68          print num_edges_added, ' edges added'
69          display_graph(G, i, new_edges)
70
71      return G
72
73
```

So, we will repeat the whole process up to this point and in case the target node is not there in this list, we will do the rest of the processing that is connecting to this node by using the function G.add_edge. So, we are going to add an edge from r node that is i to the target node right. So, the number of edges added will be implemented number of edges added plus equal to 1. And we also need to keep track of the new edges added

because, we are going to change the colour. So, we early maintain a list new is large sorry new edges new edges.append, we will add this edge and the edges basically i comma private node ok..

So, this is the new edge that is added. So, we can also keep track of the number of edges added. So, that we can check whether at every step we are adding same number of edges or not. Let us check print number of edges added. So, we will just keep a track of the number of edges added and after that we can display. So, this is one iteration over that is 1 node added with m edges. So, at this point you would like to display the graph take parameters, 3 parameters G comma the new node added is i the new edges added is where you keeping the this is new edges all right.

So, this should return the graph. So, I will return G. So, I think this function is done and we can start executing it ok.

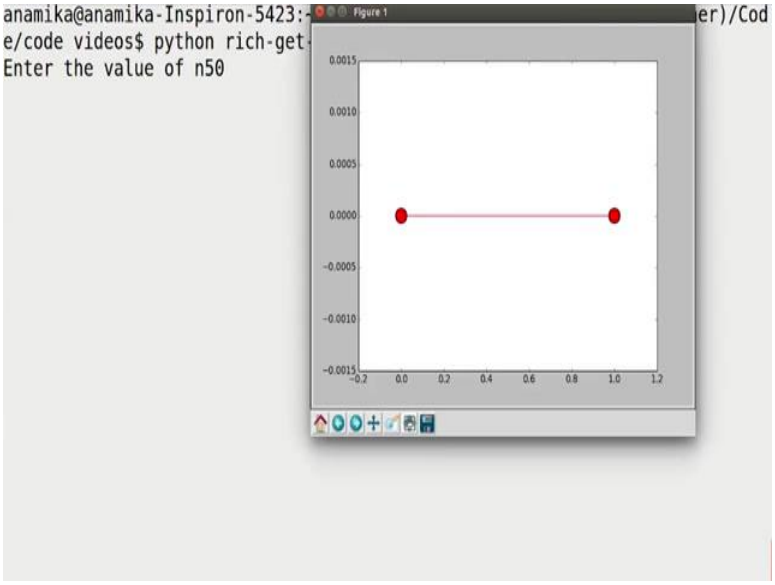
(Refer Slide Time: 27:17)



So, this should return the graph. So, I will return G. So, I think this function is done and we can start executing it ok.

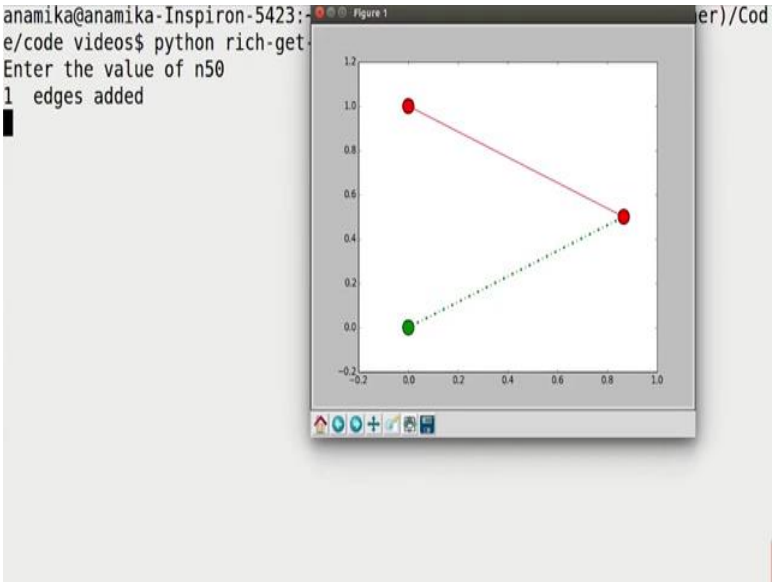
So, that is check it enter the value of n let me enter 50.

(Refer Slide Time: 27:23)



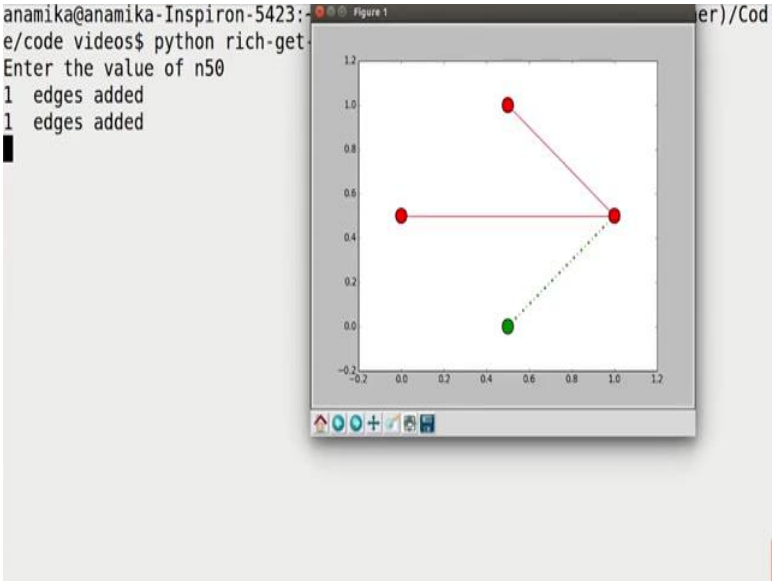
So this is the initial graph, it has chosen m0 to be true.

(Refer Slide Time: 27:32)



I will close this graph and see this is what I get in the first iteration, the new node is getting attach to one existing node. So, green is a new node and this dotted line indicates the new edges added.

(Refer Slide Time: 27:45)

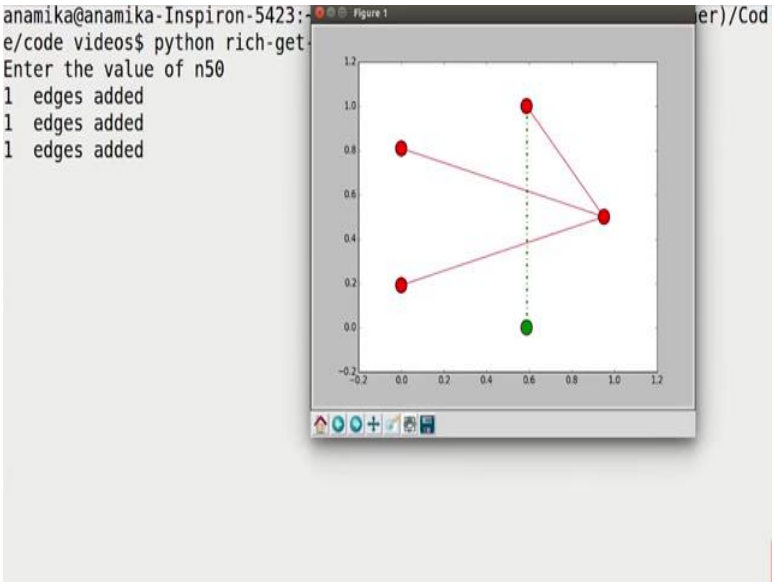


So, I am closing it and I see the second iteration the next new node is getting attach to the one of the existing edges.

So here, you can see there are 3 existing nodes, which are red and 2 of the nodes have degree 1 and o of the nodes has degree 2. So, the node which has degree 2 had more probability of getting attach to the new node, which is precisely what is happening having said that you cannot always be showing that always node, which has high degree will always big connecting it just a probabilistic phenomena right because, the referential attachment is probabilistic mechanism..

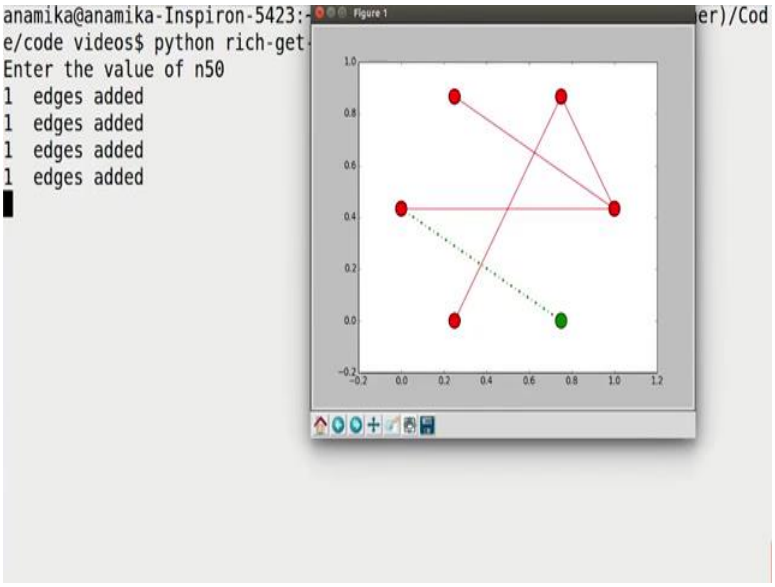
The new node is free to connect to any node in the network whether it has high degree or it has just single edge. However, if new node has a choice between degree 2 and a degree 4 node then degree 4 node will be twice as likely to be connecting to the new node as compare to the node will with degree 2. So, that is what happens.

(Refer Slide Time: 28:54)



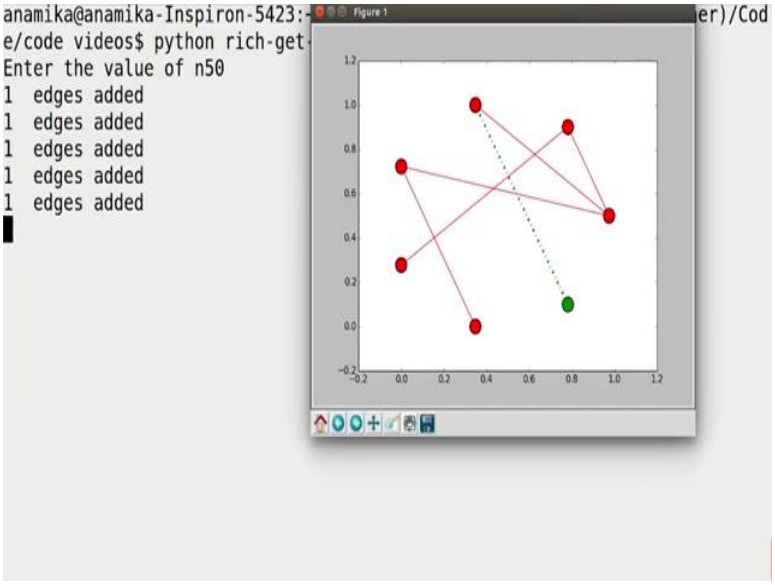
Now let us take the next iteration ok. So, the new node is getting attach to a node, which has degree 1. So, by chance it got connect to a node with less degree because, this is probabilistic. Let us check the next one.

(Refer Slide Time: 29:09)



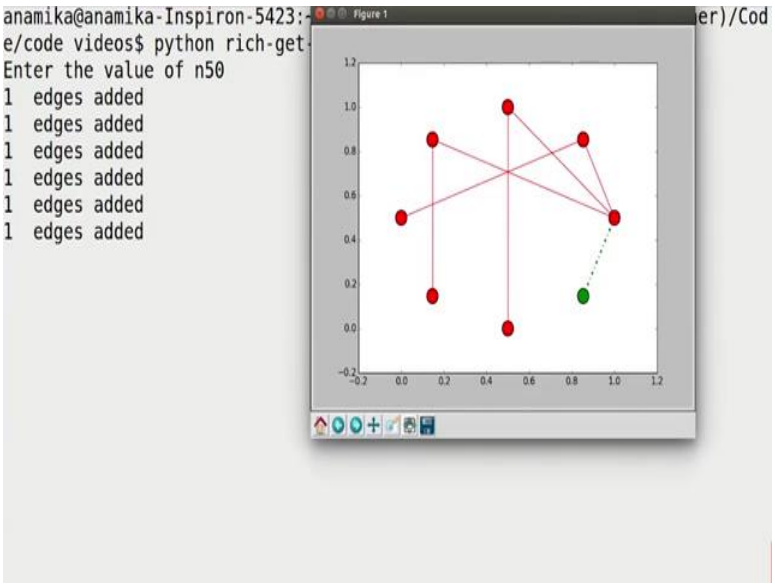
So, again the node is getting attach to a node with degree 1 because, there are few nodes.

(Refer Slide Time: 29:14)



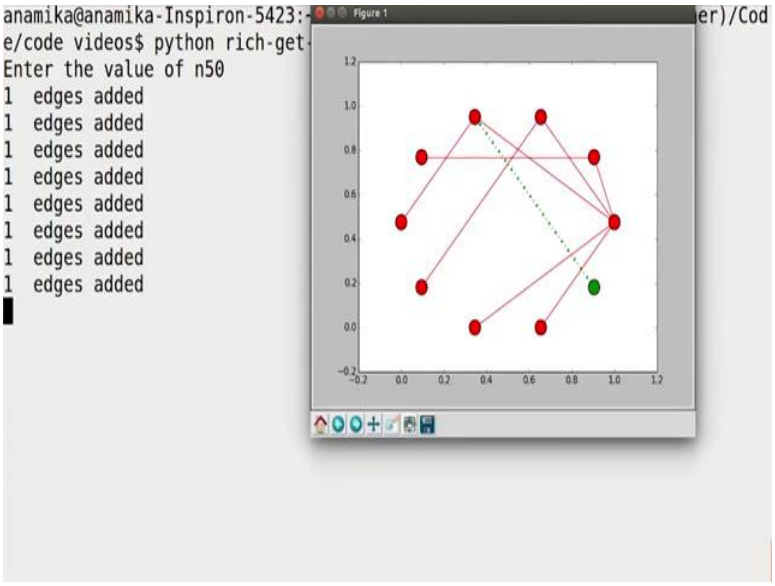
Now again the node is getting attach two by chance a node with less degree. Let us see the next one.

(Refer Slide Time: 29:20)



So now, this node has chosen an existing node high degree. Let us check the next one. Again, it has to choose the node with high degree.

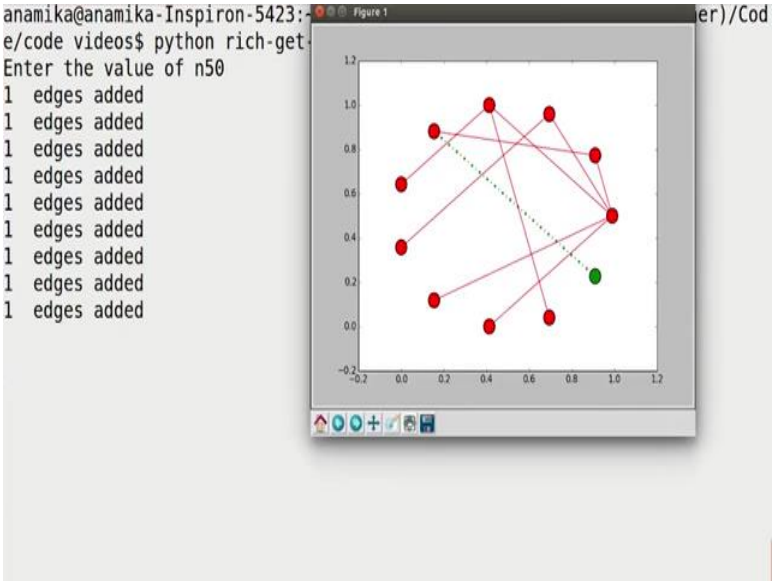
(Refer Slide Time: 29:30)



Now it has chosen our node with degree 2.

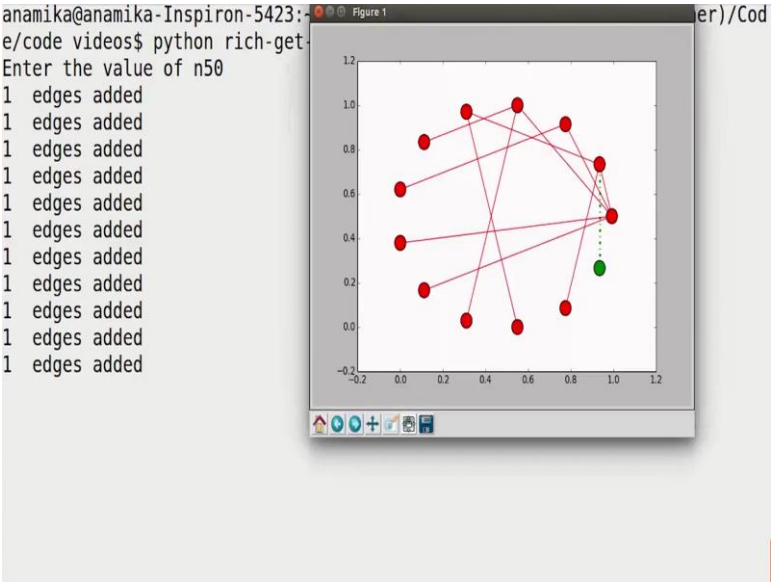
.

(Refer Slide Time: 29:34



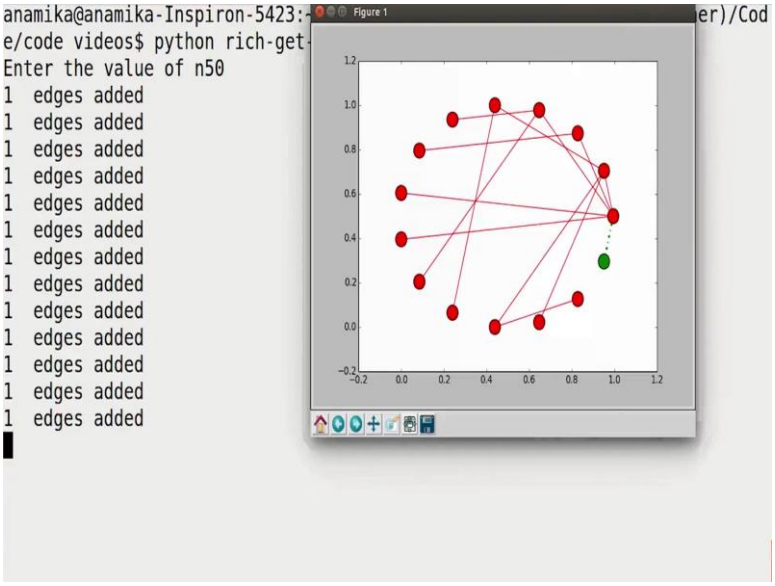
So, that keeps happening.

(Refer Slide Time: 29:36)



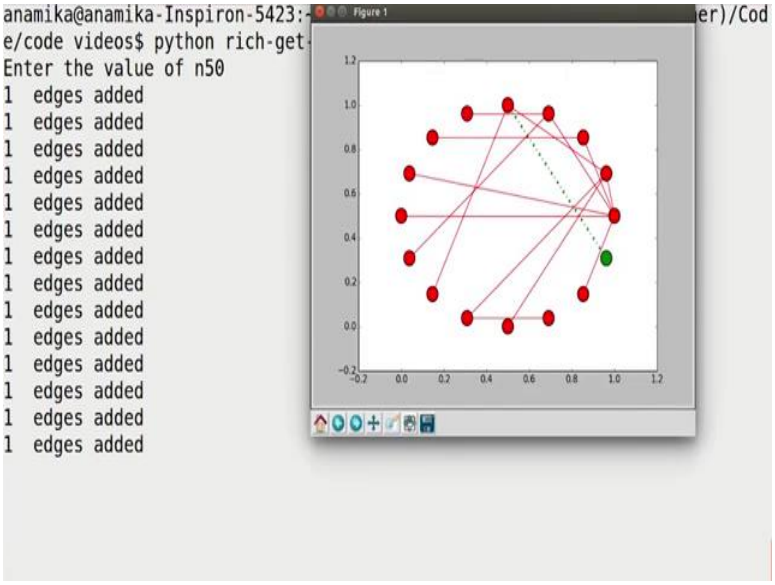
But overall, the nodes which have high degree will be connecting more to the new nodes.

(Refer Slide Time: 29:42)



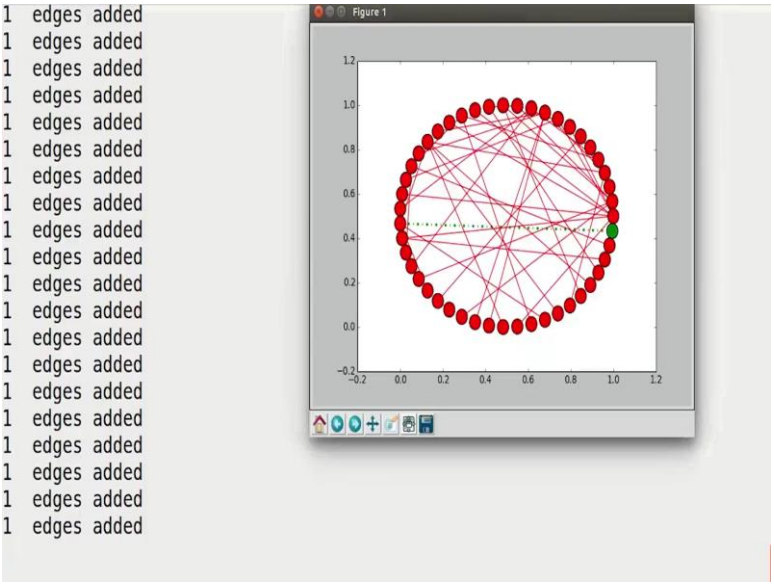
So, you can see here again happening there. So, it is again getting the connected to the node with high degree.

(Refer Slide Time: 29:47)



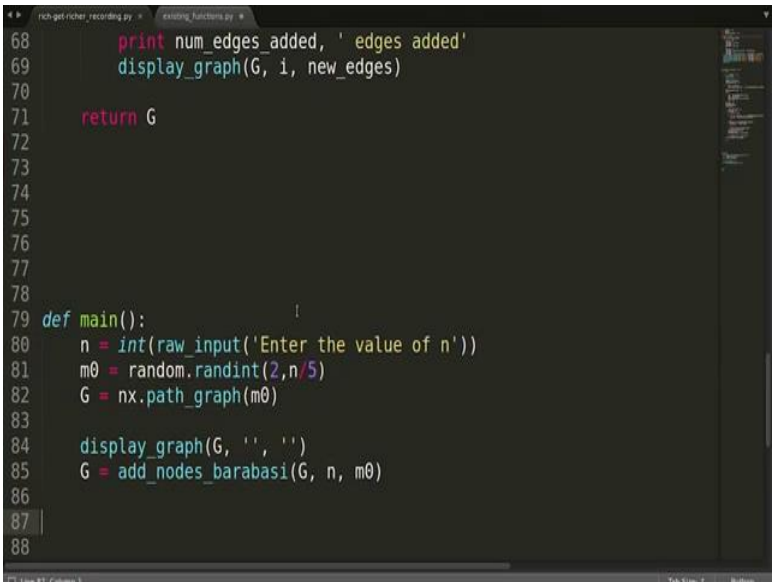
So, at each times and the network will not be symmetrical there will be more edges towards one towards some bunch of nodes, if that is not happening this network would have been symmetrical which is not so. So, I will have to keep closing this symbol 50 times. So, let me quickly do that you can keep observe with behavior ok. I should have taken the smaller network anyway.

(Refer Slide Time: 30:25)



So, this is the kind of structure that you are getting. So, a code is working fine alright.

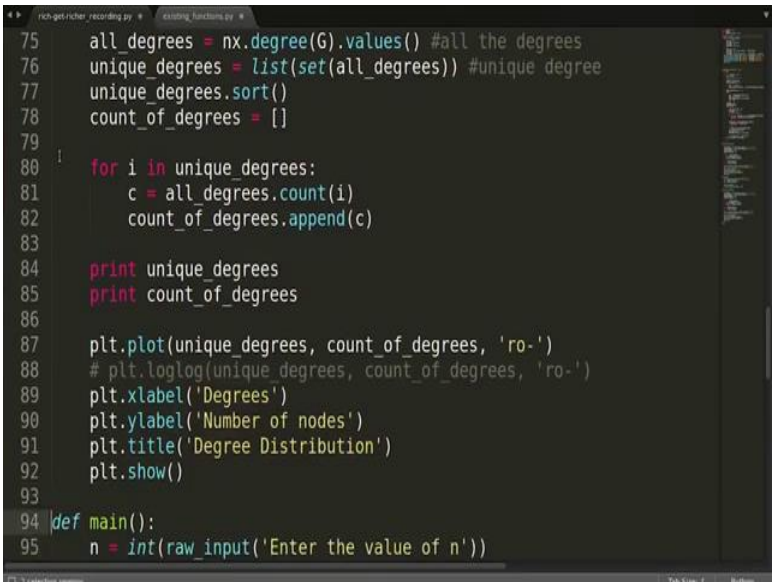
(Refer Slide Time: 30:35)



Now, an important property of the network start follows Barabasi Albert model or preferential attachment or Rich-getting-richer phenomena is that the degree distribution follows power law ok. That means, that the number of nodes, which less get less degree are way to high number and the number of nodes with very high degree are very less in number. So, that is the sort of property that is networks follow.

Let us try to get the degree distribution for this network, we had already created this function for plotting the degree distribution in one of the previous videos, I am going to just copy paste from there and I will briefly explain that. So, I will copy paste it, I will quickly explain this ok.

(Refer Slide Time: 31:24)



So, what you are doing here is since, you want to plot the degrees the distribution of the degrees, we need to get all the degrees. So, we are using the function nx.degree(G), which returns a dictionary we are taking all the values from that dictionary that is the all the degrees, we are store in this list.

Now, out of that list we are maintaining list of unique degrees because, what we have to check we have to check the number of nodes having a particular degree. So, we should know what the unique degrees are basically, the all possible degrees that the nodes have in that network. So, that is what will be kept in this list and then we are sorting this list because, we have to plot them. And we will keep a track of the number of nodes with degree 1, degree 2, degree 3 and so on, where the x axis will have all possible degrees in sorted order..

So, we have to maintain a count of degrees for all possible degree in the network. So, we have created this, list now in order to populate this list we have started this loop for i unique degrees, we will take first element of unique degree. And we will check in all degrees how many elements that is how may nodes have that particular degree; we will make use of a function count. So, we writing all_degrees.count(i), this will return as the number of nodes having the degree(i) and we are appending that to this list count of degrees and then we are simply plotting them.

So, x axis will have unique degrees, all possible degrees sorted, and y axis will have the count of degrees that is count of nodes having that particular degree. I am get displaying them in their color and this is for labeling title that is what is the function about ok.

(Refer Slide Time: 33:29)

```
85      print count_of_degrees
86
87      plt.plot(unique_degrees, count_of_degrees, 'ro-')
88      # plt.loglog(unique_degrees, count_of_degrees, 'ro-')
89      plt.xlabel('Degrees')
90      plt.ylabel('Number of nodes')
91      plt.title('Degree Distribution')
92      plt.show()
93
94  def main():
95      n = int(raw_input('Enter the value of n'))
96      m0 = random.randint(2,n/5)
97      G = nx.path_graph(m0)
98
99      # display_graph(G, '', '')
100     G = add_nodes_barabasi(G, n, m0)
101     raw_input()
102     plot_deg_dist(G)
103
104
105 main()
```

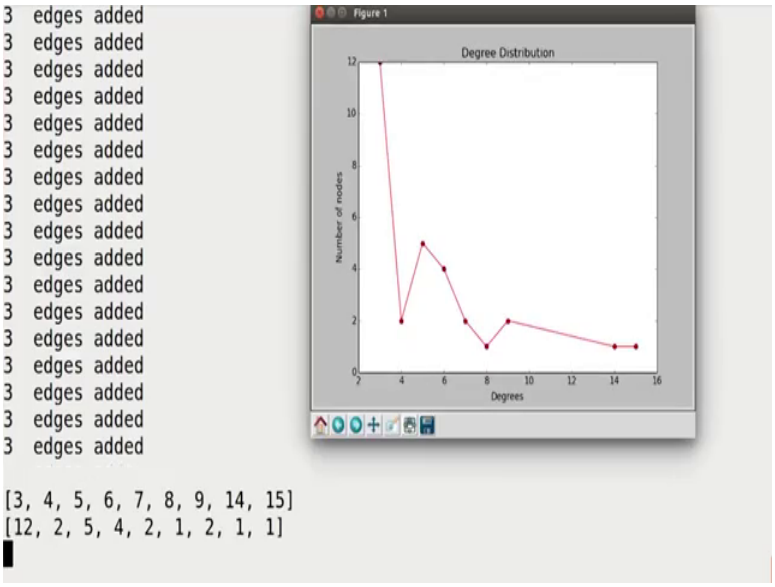
Let us call this function here i am sorry here in the end in name we are calling this function plot degree distribution G ok. So, let me let me ask the user to plus enter after getting all the plots after getting all the graphs.

(Refer Slide Time: 33:54)

```
anamika@anamika-Inspiron-5423:~/NPTEL/WEEK_wise/WEEK_7_(Rich get richer)/Cod
e/code videos$ python rich-get-richer_recording.py
Enter the value of n
```

Let us check the functionality I will get. So, 30 this is initial graph, and these are the nodes getting added I have to press this 30 times and after that we should get the distribution get everything works fine ok.

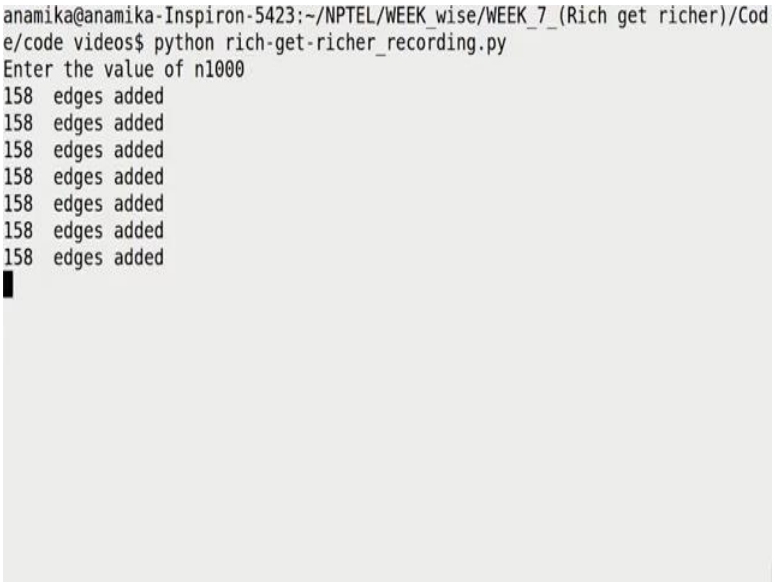
(Refer Slide Time: 34:16)



I press enter now this is the distribution that we are getting. So, this is sort power law in the sense that the nodes with less degree are very high number and the nodes with very high degree are very less in number. So, power law is basically the sort of plot since a number of nodes is less. We are not getting it very nicely calling power law, but nevertheless it is following the that law.

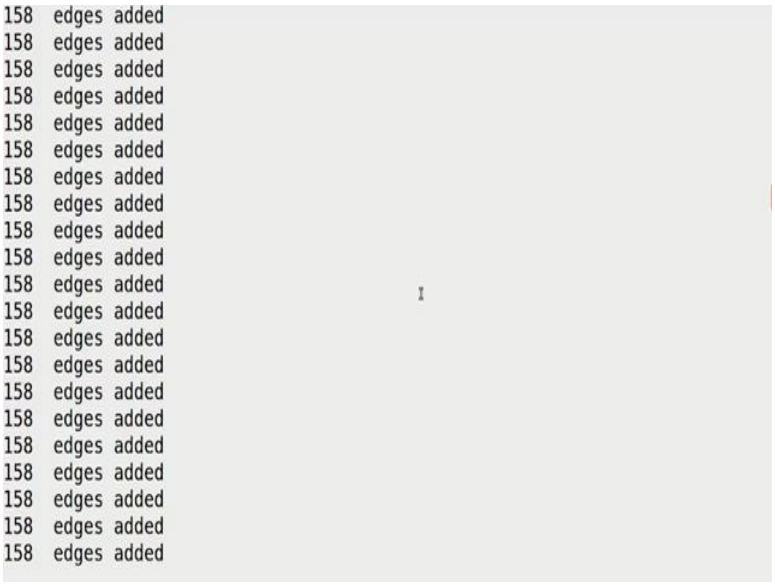
If you take good number of nodes say some 100 or say 1000, you will get a nice distribution you can also check that press check that. So, let us check let us execute this code for some higher value of n. So that, we can see the distribution nicely so in order to that I need not press the I need not close the graph multiple times, I am going to remove this displaying of the graph from all the places that is all that is that is only we will calling this display graph function. So, I have commented that.

(Refer Slide Time: 35:27)



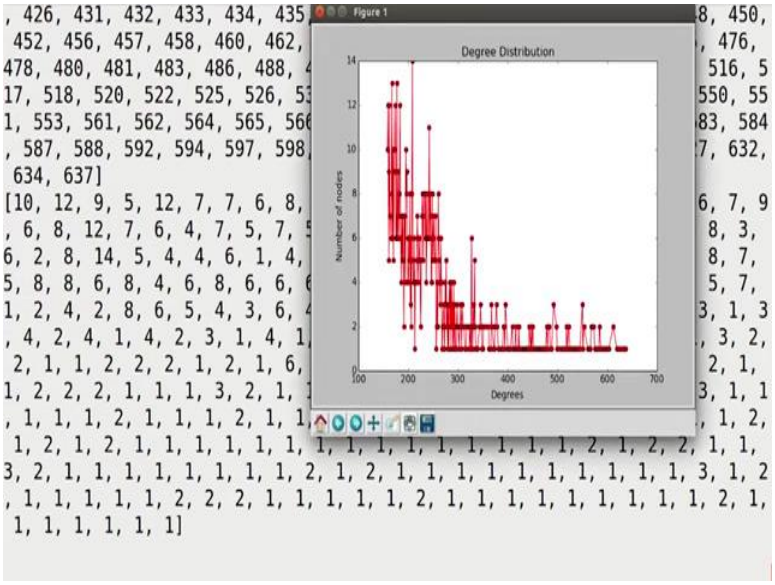
And let us call this with a high value of n say 1000.

(Refer Slide Time: 35:33)



So, 158 edges are getting added at every movement I think it is a very high number, I think, I should fix it on a smaller number there this is going on and on because, it has to go on for 1000 times. So, it is going on and on it is taking some chain let us see it is just finished, I am sorry.

(Refer Slide Time: 35:53)



So, I will press enter and I got the distribution. So, this is fall in solve power low as you can see right if you take a log plot of this, we will get this straight line. So, that is the whole idea about this.

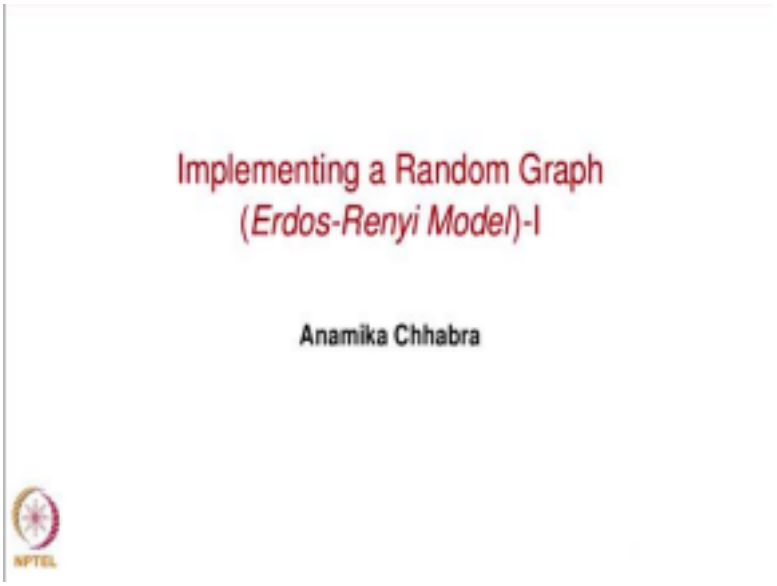
So, as I told you the networks which follow preferential attachment follow power law as well and that is what we have observe there is well. So, that was the whole idea behind Rich-getting-richer phenomena. Here, we were adding the edges based on the degrees right, more degree more probability of getting connected. On the other hand, if you add the edges randomly what should happen? in that case, you will not get this power law distribution because, there is no preference to words getting a attach to a particular node, we are just randomly getting connected to the existing nodes.

So, in that case you will not the getting power law. In fact, you will getting normal distribution. So, in the next video we are going to implement random networks and we will see the kind of distribution that we obtain.

Social Network
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Rich Get Richer Phenomenon
Lecture - 123
Implementing a Random Graph (Erdos– Renyi Model) - 1

(Refer Slide Time: 00:05)



Hey everyone, in the previous video we implemented Barabasi-Albert model which follows preferential attachment that is also called rich getting richer phenomena. In this video sequence we are going to implement Erdos Renyi model which is used to create random networks. In this model each edge has a fixed probability of being present or absent independent of the other edges.

The idea is that we will start with the network with some n nodes with no edges amongst them. Now there are going to be ${}^{n}C_2$ pairs of nodes in the network. We have to decide whether which whether an edge should be added between the nodes of each of these pairs or not.

Now, let me explain that method to you using example. So, we will take the first pair of nodes and we will take a coin we will toss it. If we get a head, we will put an edge between this pair of nodes, if we get a tail, we will not put an edge between this pair of

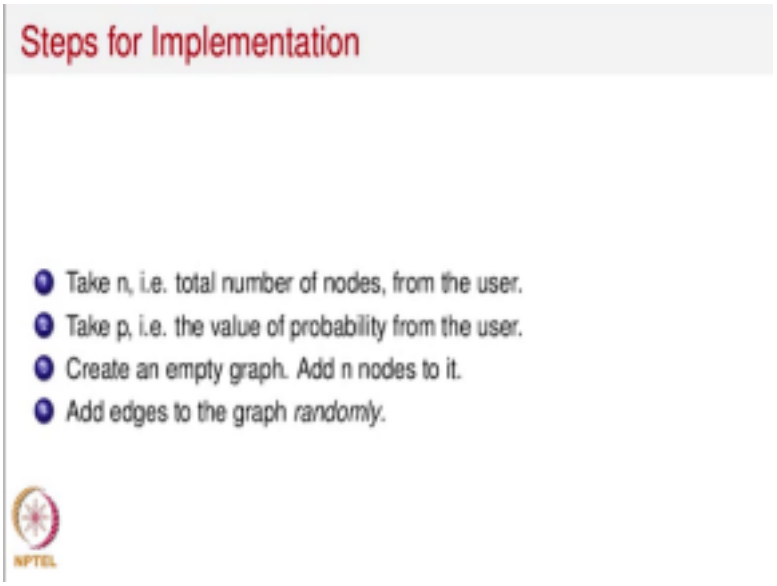
nodes. Then will take the next pair of nodes and we will toss the coin and we will do the same.

This we will keep doing for each pair of nodes in the network which is ${}^{n}C_2$. So, in the end we would have added some number of edges in the network. To generalize and to understand how it actually happens in the network in the model, we basically take some value of p. So, when we toss a coin which is not biased p is equal to 0.5 that is you get the head and tail with equal probability.

However, in the model we take the value of p from the user or we fix it this value, this can be any value between 0 and 1. So, for example, assume p = 0.3. In that case if you if you correlate with the example that I gave, the coin will be biased with the probability 0.3 that is head will appear with probability 0.3 and tail will appear with probability 0.7 ok.

So, that's the whole idea and you can imagine that if the value of p is high, the more number of edges will be added to the network and if the value of p is less than less number of edges will be added in the network, if value of p = 0 no edges will get added. If the value of p = 1, then all the edges will be added to the network and it will be a complete graph. So, that is the whole idea behind this Erdos Renyi model. Let us look at this steps that we are going to follow for the implementation.

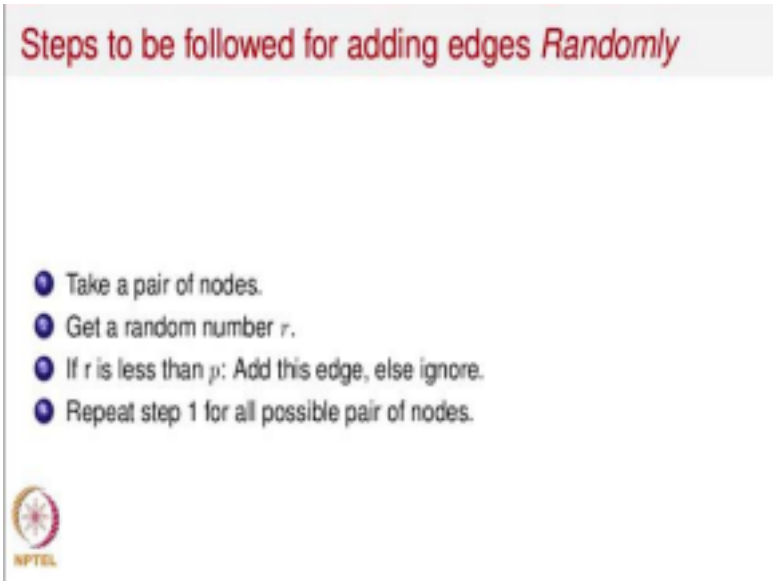
(Refer Slide Time: 02:47)



So, there are 2 parameters that are required for implementing this model that is n and p, where n is the total number of nodes in the network and p is the value of probability as I told you, this will be input to the model and based on that only the edges will be added. So, we are going to take value of n from the user and then we will be taking the value of p from the user.

So, initially the network has only the nodes. So, we will take an empty graph and we will add n nodes to it and there are no edges in the network as of now. So, edges will be added to the network randomly Now, let me explain you how we are going to randomly add the edges.

(Refer Slide Time: 03:34)



So, we are going to take a pair of nodes ok. Basically we have to process all the pairs of nodes which is ${}^{n}C_2$ pairs, we will take the first pair and we will get a random number r between 0 and 1. If the value of r <= p we will add an edge between this pair of nodes and if value of r > p we will not add an edge between this pair of nodes and we will check the next one. We will keep repeating this step 1; step 1 2 3 actually ok.

We will keep repeating the first 3 steps for all possible pair of nodes and by the end after we have processed all the pairs, we would have added certain number of edges to the network. So, this is the whole idea that we are going to follow for the implementation in the next video and we will also see the degree distribution that this kind of network

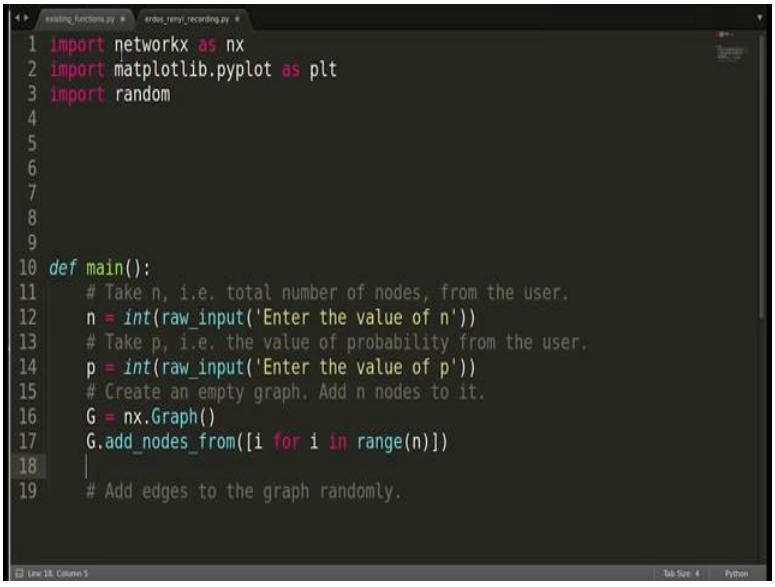
follows.

Social Networks
Prof. S. R. S. Iyengar
Prof. Anamika Chhabra
Department of Computer Science
Indian Institute of Technology, Ropar

Rich Get Richer Phenomenon
Lecture - 124
Implementing a Random Graph (Erdos- Renyi Model)-2

In the previous video we discussed Erdos Renyi model to create Random networks. We also discussed the sequence of steps that we will be following for the implementation. In this video we will do the implementation.

(Refer Slide Time: 00:21)



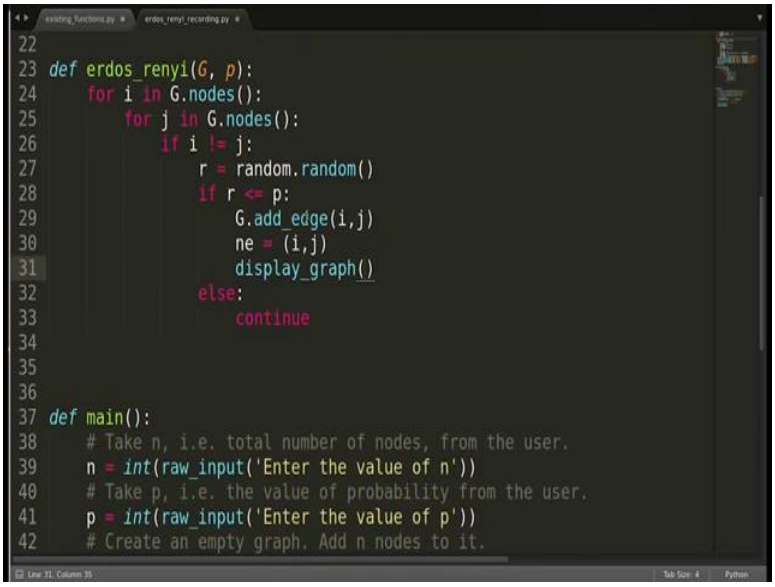
I have pasted the sequence of steps that we discussed in the previous video over here and we will start implementing them. To start with let me import the packages that we are going to need. We will be needing networkx, I am sorry we need to display the graph, so we will need matplotlib. We also need random package ok. Let us start the main. Let us take these steps 1 by 1. So, we have to take the value of n from the user and also we have to take the value of p that is probability from the user.

We can use the function draw input for that. So, we can write n is equal to draw input returns in string format. So, we need to convert it to int. Similarly, we will take the value of probability ok. The next step is to create an empty graph and add n nodes to it. Will

create an empty graph, we have to now add nodes to it n nodes to it, we will use the function G.add_nodes from here we can pass a list. So, we must add n nodes, what we can do is will write i from i for i in range n is the total number of nodes.

So, 0 to n - 1 nodes will be added to the graph. At this point we can also display the graph we had already created this (Refer Time: 02:49) graph function in the previous video. So, we are going to reuse it. Let me copy paste that function from the previous videos content and we will use it as it is.

(Refer Slide Time: 03:05)



So, initially when we display the graph it be all empty only the nodes will be there. The next step is to add the edges into this graph which only has nodes as of now, we will create a function for that.

We have to add the edges randomly as per Erdos Renyi model. So, let us create that function. Will pass the graph and the other thing that must be passed is p. So, in Erdos Renyi basically we pass 2 parameters n and p. We pass n here or we can compute the number of nodes from the graph itself. So, I am going to pass only these 2 parameters.

Let us create this function, now, if you remember from the previous video, we have to add the edges randomly. So, basically, we will take one pair of nodes and based on the probability p we will add an edge, or we will skip it.

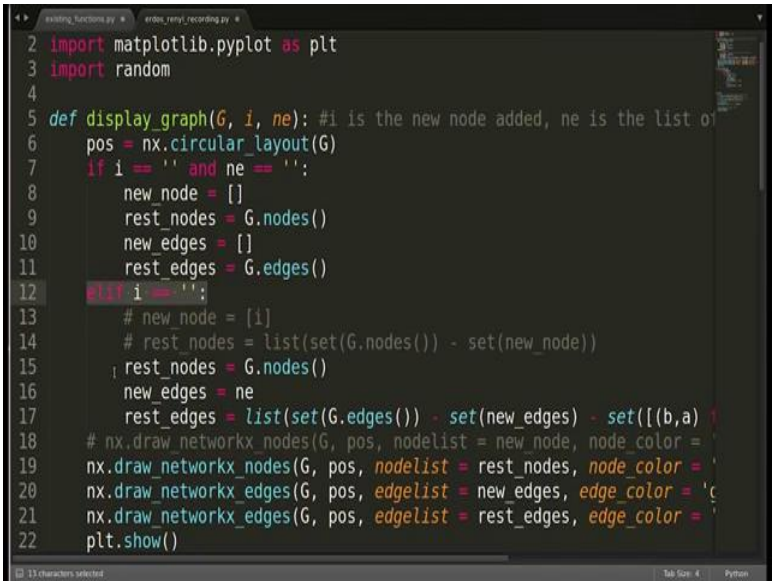
So, let us take all pair of nodes, how can we do that? We will start we will use 2 for loops write for i in G.nodes. We got all the nodes here as the first element in the pair and we will write for j in G.nodes. We do not have to add the edges to from we do not have to add the self edges from one node to itself.

So, we will put a condition here, if i != j only then we will add the edge. We have to take a random number here and will compare that random number with probability p. So, we will take r = random.random, which will return us a value between 0 and 1.

So, if r <= p, we will add that edge. So, we will write G.add_edge. The edge is going to be i comma j and in case r is greater than p we will not add that edge. So, we will just write continue. So, these are the main things to add the edges. Now we also want to display the graph and we also want to keep track of the edges which are getting added at each iteration. So, let me do one more thing here.

So, we will keep track of the edges which are being added into this variable i comma j ok. And then I would like to display the graph, so let me call that function display graph which we have already added. Display graph has two parameter 3 parameters the graph as you can see here, the graph and the node which is being added and the edge which is being added.

(Refer Slide Time: 05:58)



So, here nodes are not being added because, all the nodes are added to the graph initially only the edges are getting added at each iteration. So, if you note here and if you remember ne is a list of edges right and i is a single node.

(Refer Slide Time: 06:24)

```
        for j in G.nodes():
            if i != j:
                r = random.random()
                if r <= p:
                    G.add_edge(i,j)
                    ne = (i,j)
                    display_graph(G, )
                else:
                    continue



def main():
    # Take n, i.e. total number of nodes, from the user.
    n = int(raw_input('Enter the value of n'))
    # Take p, i.e. the value of probability from the user.
    p = int(raw_input('Enter the value of p'))
    # Create an empty graph. Add n nodes to it.
    G = nx.Graph()
    G.add_nodes_from([i for i in range(n)])

```

So, we have to keep we have to take care of that while we are passing the things here. Now we do not want to change the colour of the nodes, we will just pass an empty string here and we have to pass the edge which is being added in this iteration as a list. So, let me convert this into list and will what I will be passing.

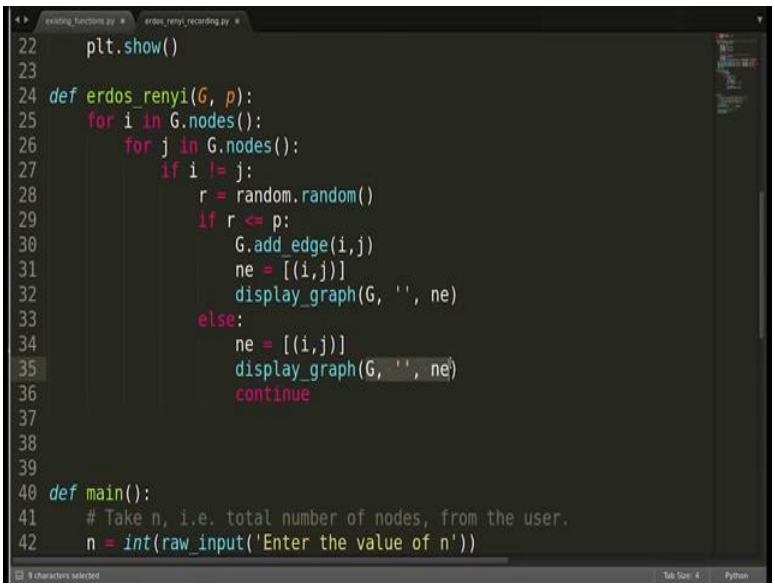
And in case we are not adding any edge we want to display the graph at that moment as well because, we want to see whether in this particular iteration any edge was added or not, we want to display the graph there also. So, let me copy this in that case there will not be any new edge. So, we will keep this empty and everything else should remain same. Now let us go back to main and see if we can start executing this code.

(Refer Slide Time: 07:19)



We are calling this function Erdos Renyi, we are calling display graph. There are 3 parameters to this function and in main we are passing only 1 parameter. So, let me pass the rest 2 parameters. So, this is the initial displaying of the graph where there are no new nodes, no new edges. Let us pass the empty strings there. So, the initial graph should be created.

(Refer Slide Time: 07:46)



And when we are calling display graph from this function, we are passing G comma, we passing G and we passing empty string and a list ok. Let us see whether this function

were surprise for that. So, the initial condition this condition cannot be used because, both of these are not empty and if you go to the else part or i is empty here.

So, we see that we need to make some changes into this function in order to be able to use this for our set of conditions in this particular model. So, maybe we can write elif i = empty because this is the case that will be followed when we call display graph from Erdos Renyi function. Now we basically do not know want to change the colours of nodes because, every node is added initially. So, maybe we can let go off this thing ok.

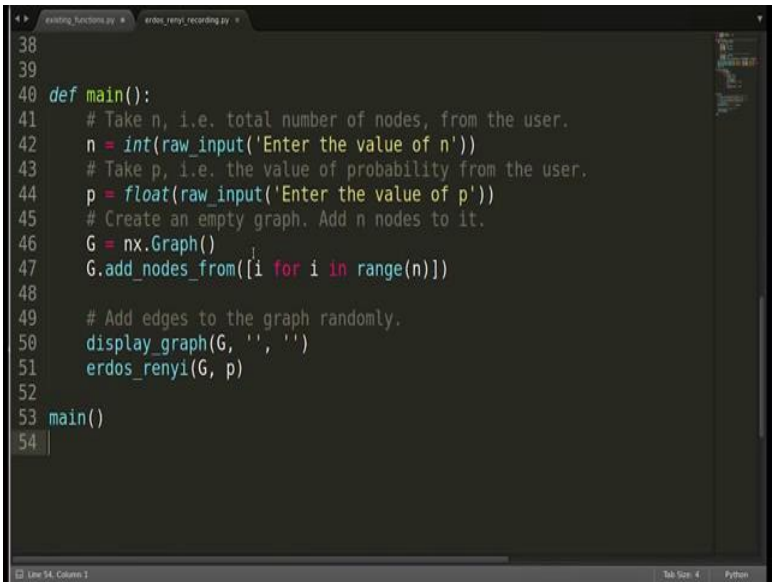
Only the edges coloured has to be changed. So, this is fine, and this is fine and this should be fine let me see, we want to display the nodes right. So, let me remove the new nodes statement from here and the rest of the nodes will be displayed in red colour. Rest of the nodes is what we have to define here.

So, I will write rest and I will just copy from here ok. I am just playing around you will be able to do it yourself as well or you can write the function once again. I will just explain after I am done with this. I think they should do ok.

So, in our case when we will calling display graph function from the set of Renyi function, we are passing 3 things and in all the cases G is there and empty string is passed as i and the third one is the list. In case we are not adding the edge, this list is going to be empty.

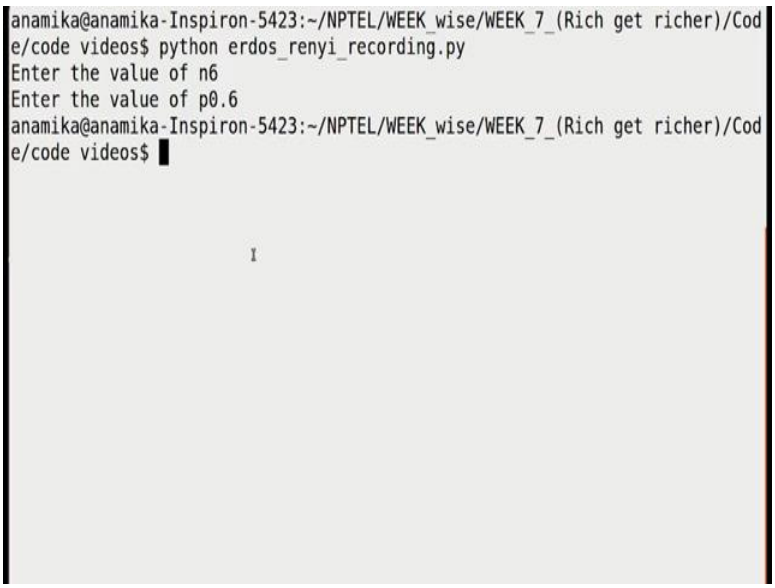
So, let us see which condition will be followed. This condition will not be followed because, ne is not empty. We will be going into this and rest of the nodes is are going to be all the nodes and new edges yes we are passing a list there, yes this should work.

(Refer Slide Time: 10:27)



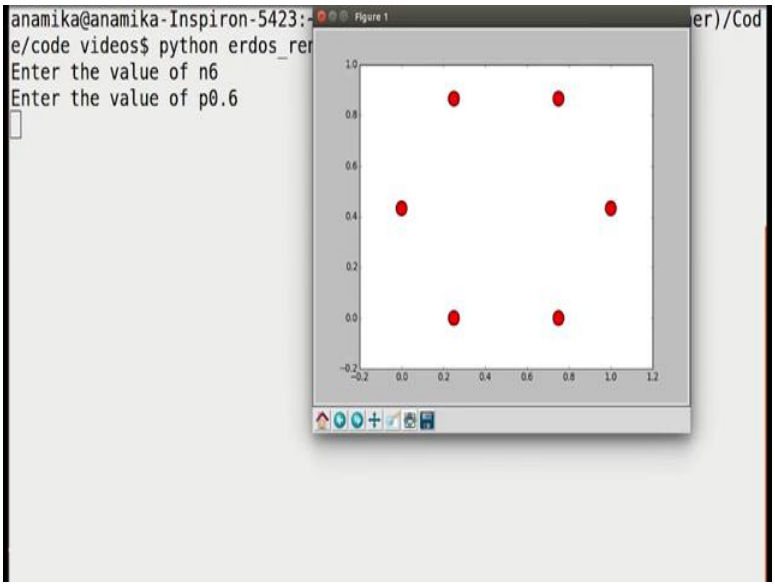
Let us go to the main and the value of n is going to be integer right and the value of probability is going to be between 0 and 1. So, that is actually no going to be integer. So, let us add a float there. Now let us call this main function. Now we can now check the functionality of this code.

(Refer Slide Time: 10:53)



So, let us check how it works. Enter the value of n, so let me pass a small value so that we can check the graphs at every iteration let me pass 6 for example. The value of p let me add 0.6 here we can pass any value between 0 and 1.

(Refer Slide Time: 11:15)

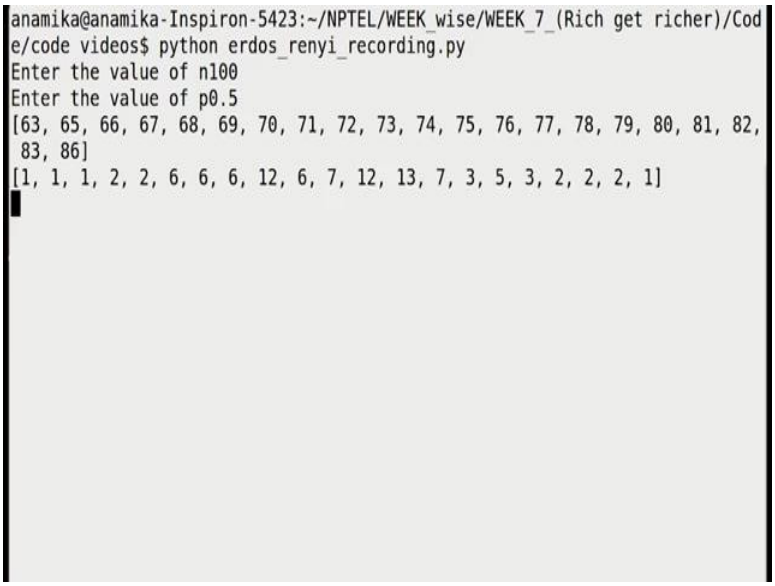


So, this is the initial graph where there are only nodes, 6 nodes are there, there are no edges. Now I am closing this and in the next iteration we are getting this edge between these 2 nodes and in the next iteration again we are getting this edge, we are getting an edge and in the 4th iterations as you can see we are not getting any edge. We got this edge and so in some iterations as you can see, we are getting some edges one edge and in some iterations, we are not getting.

So, I think I have to do this for 6 into 6, 36 times right. So, as you can see 36 times this loop will run and in some of the iterations i might be equal to j, so no edge might be added. In some of the iterations the r might be greater than p, so h might not be added. So, this was the functioning of Erdos Renyi model that we created. I also told you that the degree distribution that Erdos Renyi model follows is not power law rather it is normal distribution. We had created a function for plotting the degree distribution in one of previous video.

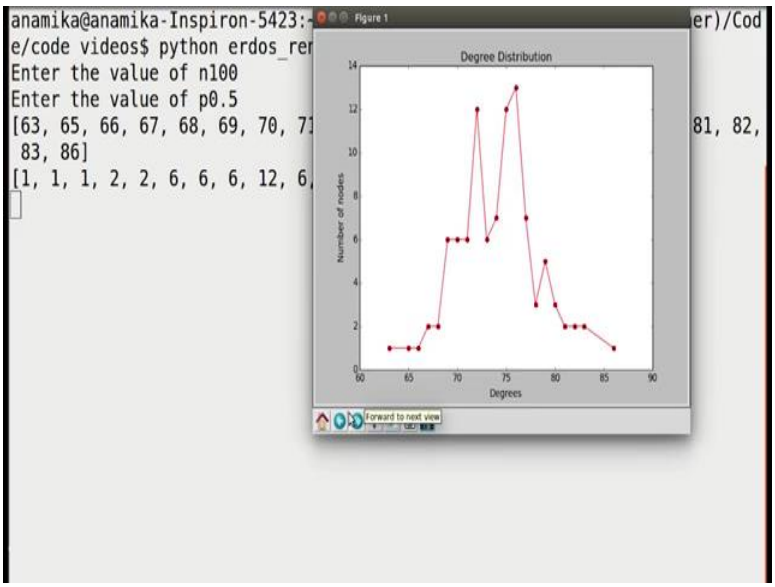
So, I am going to just use that function as it is, let me copy that and we will use this function plot, degree distribution, so let us use it here. In order to show you the degree distribution I will take some good number of nodes so that we can see a nice plot with a with many values. So, I will remove the displaying of the graph so that, I do not need to press all tab 4 again and again. So, I am just commenting this displaying of graph here. Now let us check the functionality of this code.

(Refer Slide Time: 13:23)



So, we have to observe the degree distribution for an Erdos Renyi network. Let me add 100 nodes and let me add the probability to be 0.5.

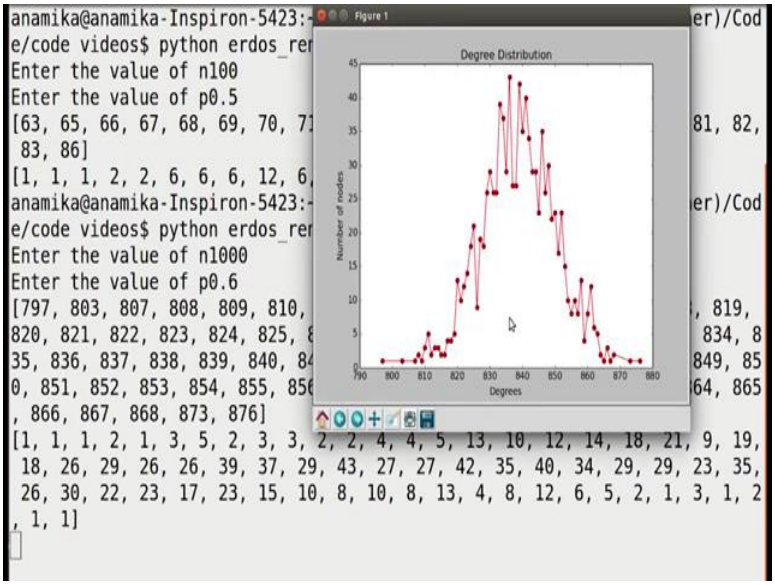
(Refer Slide Time: 13:33)



So, this is the sort of curve that we obtain. So, you can see since we were adding the edges randomly, there were no preferential attachment. We can see that there are very less nodes with very less degree and there are very less nodes with very high degree and there are very large number of nodes with medium degree. So, that is a sort of

distribution that we obtain, sort of normal distribution that we get in the case of a random network.

(Refer Slide Time: 14:08)



Let me try it once again for a greater number of nodes. Maybe I will add some 1000 nodes and the probability to be 0.6. So, again this is nice curve that we obtain. So, as you can see as you keep increasing the number of nodes, you can see the nice distribution that it takes ok. So, that was about the Erdos Renyi network to Erdos Renyi model to create the random networks and it follows the normal distribution.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Rich Get Richer Phenomenon
Lecture - 125
Forced Versus Random Removal of Nodes (Attack Survivability)

In this video we are going to implement Forced Versus Random Failure of Nodes which is also called Attack Survivability. We are going to take a network and we will be removing nodes from this network and we will observe the effect of removal of nodes from the network. We will be using two strategies to remove the nodes. In the first strategy we will remove the nodes randomly, and in the second strategy we will remove the nodes which are highly connected.

We will observe that when we remove the nodes which are highly connected, it requires very less iterations for the graph to become disconnected. This sort of information can be used by some agent for example, if he has the information of who is more connected and who is not. He, if in case he wants to damage the network for example, by spreading some sort of virus, he will not be attacking a random nodes rather, he will attack the nodes that are more connected, so that the network becomes infected to a greater extent.

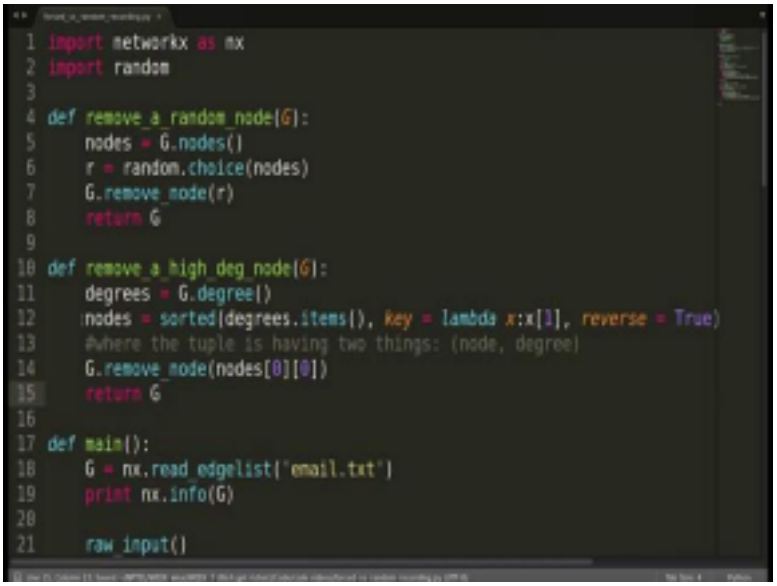
So, there is one thing we will observe and implement, and we will also see one more thing in the context of the kinds of graphs. For example, the first thing that we checked on real world networks, real world networks sort of follow preferential attachment where there are there is existence of hubs in the network which are responsible for connection of most part of the network. So, when you remove then the graph becomes, becomes disconnected.

We are going to observe the same thing that is the removal of nodes on random networks as well. And we will observe that it does not make much difference on a random network when we remove the nodes randomly as compared to when we remove the nodes selectively; that is we remove the nodes which are having high degree.

That is because there is there are no hubs in random networks which are responsible for

the connection of most part of the network; so the number of iterations that it will require in the first case that is random removal and the second case that is selective removal. In the case of random networks will not make much difference. We are going to implement that and we will observe the number iterations that are required. Let us get started with implementation.

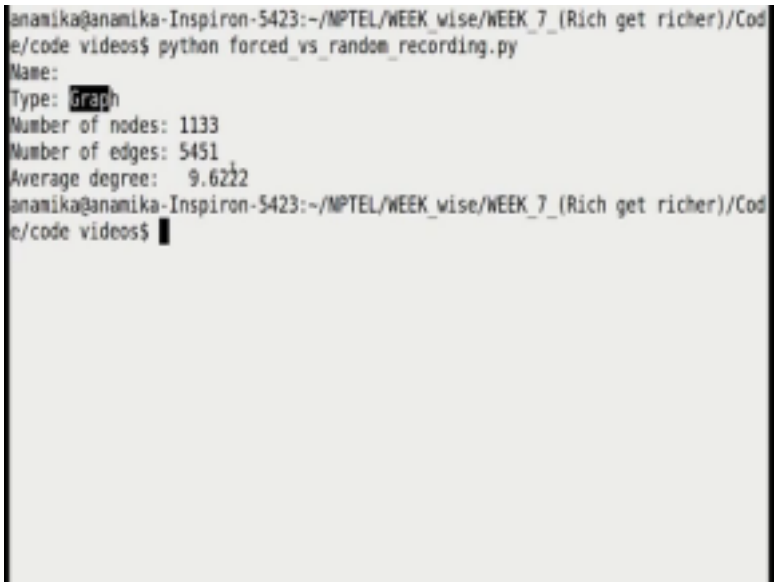
(Refer Slide Time: 02:23)



Let me import the packages first, we might also need random package, let us define our main. I already have downloaded a real-world network that is email network and that is an edge list format, we will be making use of that. This network is basically undirected. So, there is link from a node a to node b, if they exchanged any email with each other. So, that sort of network is what we will be using.

Let me read that network n x.read_edgelist email.txt. Ok I have kept this in the same folder where I have get this file and then let me print some details about this network. So, I will write n x.info, so that I get the basic statistics about the graph. Let me call this function let us now check whether the graph has been read or not it has been converted into graph object or not.

(Refer Slide Time: 03:46)



So, this is the basic statistics about the graph. It has more than 1000 nodes and more than 5000 edges and the average degree is there. So, let us get back here. So, we are going to remove the nodes from this network in 2 ways: let me print the first page random removal ok. We are going to randomly remove the nodes. Let me create a copy of the given graph because, we are going to remove nodes in 2 ways.

So, we will work on both the copies. So, this is the first copy G1 from which we will remove the nodes randomly. First let us check whether the graph is connected or not, is G1 connected ok, I will write nx.is_connected(G1) ok. We have to keep a track of how many nodes have to be removed to make the graph disconnected when we randomly remove.

So, let us create variable nodes removed when we randomly removed initialise to 0. Now while the network becomes disconnected, we have to keep removing the nodes. So, we will write nx.is_connected(G1), we have to keep removing. So, we should better create a function for removing a node I create that function in a while. I will write G1 is equal to remove a random node.

So, this function I will just create, I am going to pass the graph here ok. And as you keep removing we will increment the counter for nodes remove random plus is equal to one and in every iteration we will keep checking whether graph is connected or not we will actually print that or let me just copy paste this sorry ok.

So, we at every iteration we are going to check whether graph is disconnected or not and in the end when we are done we will come out this loop and the graph becomes disconnected and at that point of time will print the number of nodes that needed to be removed to make the graph disconnected. Randomly chosen we will print nodes remove random ok. We will lastly used to press an enter here, as well as here ok. We are going to remove the in the second strategy, we are going to remove the nodes selectively based on the degree.

(Refer Slide Time: 07:06)

```
        nodes_removed_random += 1
        print 'Is G1 connected?', nx.is_connected(G1)
    print 'nodes removed when randomly chosen', nodes_removed_random

    raw_input()

    print '###Selective removal###'
    G2 = G.copy()
    print 'Is G2 connected?', nx.is_connected(G2)
    nodes_removed_selective = 0

    while(nx.is_connected(G2)):
        G2 = remove_a_high_deg_node(G2)
        nodes_removed_selective += 1
        print 'Is G2 connected?', nx.is_connected(G2)
    print 'nodes removed when selectively chosen', nodes_removed_select

main()
```

So, let me just copy paste this entire thing and I will just made changes into that. So, in the second strategy, we are removing selectively, selective removal we will create another copy G2 of the graph. We will see whether G2 is connected or not. So, I am just making changes there. Nodes removed when selective removal was there. Let me write selectively, selective and again G2 and here also G2 and nodes remove the selective is G2 connected.

Actually, should I have written G only here, anyway nodes removed when random when selectively chosen. Nodes removed selectively right; I think there is a mistake here. Ok the only thing left is now to create this function that is 2, these 2 functions we have very minute, we have to remove the node selectively ok. So, remove node remove high degree node may be ok. So, we are yet to create these 2 functions, the first function is remove a random node and the second function is remove a high degree node.

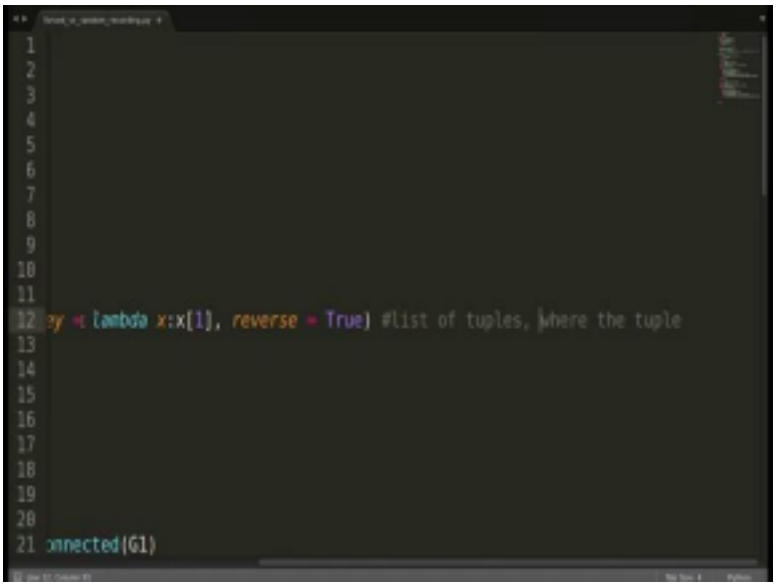
Let us create these functions ok. This function we have to create. So, out of all the nodes we have to choose 1 node randomly and we have to remove that. Let me get a list of all the nodes. nodes is equal to G.nodes, I have to randomly remove a node out of this while use random or choice out of nodes and we have to remove it. So, I will write G. remove node, which node to be removed or has to be removed and at this point we have remove the nodes from the graph and we will return it ok. Let us make a generalized just keep G here.

So, we pass the G and we return the G. So, whichever graph was input 1 more was removed, and it was returned. So, that function is being used here, everything seems fine, this syntax and return type, everything is fine. Next thing is to create this function remove a high degree node. Let us get this function now ok. What we have to do here? We have to see the degrees of the nodes and we have to sort them accordingly and the node which has highest degree will be removed.

How do we do this? Firstly, let us get dictionary having the degrees of all the nodes. So, G.degree ok, now, now out of this dictionary we have to get a sorted list of the key value pairs right. So, what we can do is a these are basically not nodes these are edges because, when we sort a dictionary we get list of tuples where every tuple is every tuple is actually key and value where key is node and the value is degree, so it make sense to call it nodes.

So, what we can do is we can use sorted function and we will pass degree degrees degree might be key or because, degree is the name of the function, so let us call it degrees. deg.items_key = lambda the same old method that we used to get a sorted dictionary. And then X, we want to sort based on the value and we want to sort in descending order reverse = true, this should work I am sorry.

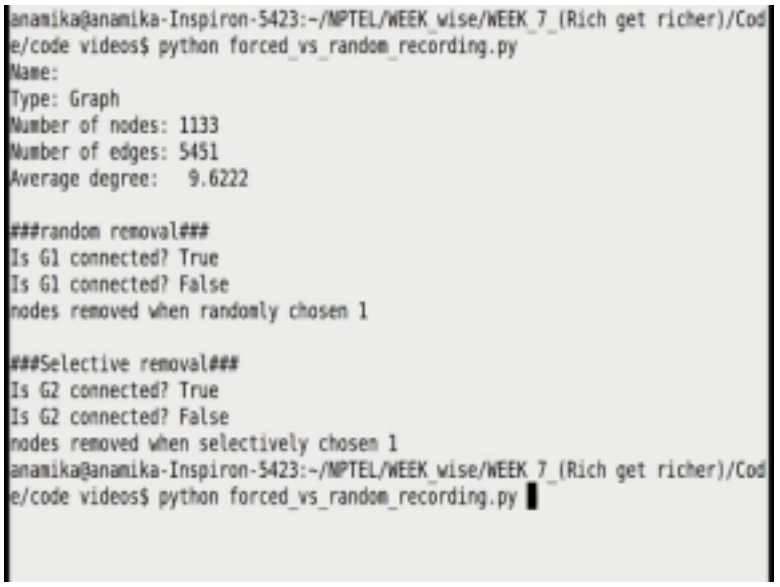
(Refer Slide Time: 11:32)



So, what is nodes? Nodes basically is list of tuples where the tuple (Refer Time: 11:41) percent is here where the tuple is having 2 things node and the yes node and the degree right and it is sorted. So, the first node will be having the highest degree and that is what we should remove.

So, I will write G.remove_node, which node we have to remove out of the nodes list we have to take the first entry that be a tuple and out of that tuple we have to take the first value that is going to be the node that has to be removed right. So now, let us return G ok. This should work it is it no mistake. So, we are calling that function here G2 is equal to remove a high degree node and we are incrementing the counter and we are connecting, that is all. It should work let us check let us check.

(Refer Slide Time: 12:43)



So, this is a basic statistics as we saw earlier too, I am pressing enter ok. So, in this case we are randomly removing, and we have to remove only 1 node to make the graph disconnected. Now I am pressing enter and we are now selectively removing and again we need here only one node to make a graph disconnected.

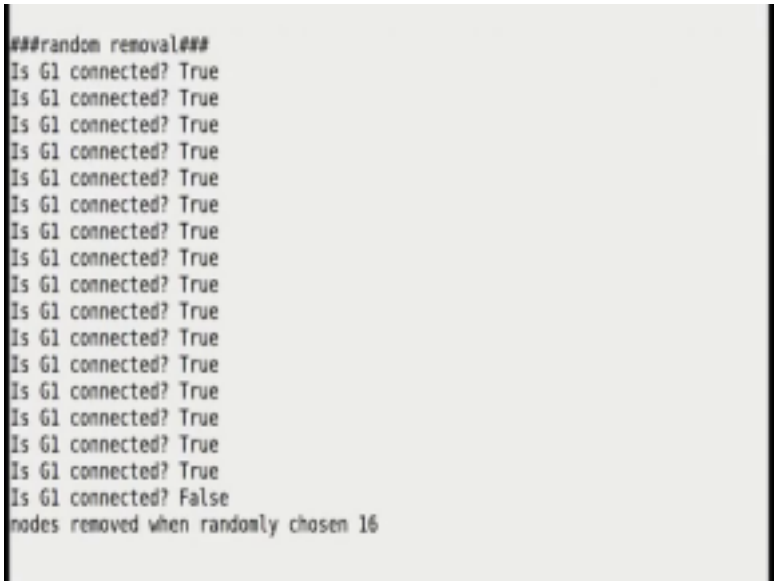
So, by chance in this case as you see whatever node we removed randomly that was by chance a high degree node that is that is how the graph becomes disconnected. Let me run this function again and let me see again it is taking one it is surprising.

(Refer Slide Time: 13:26)



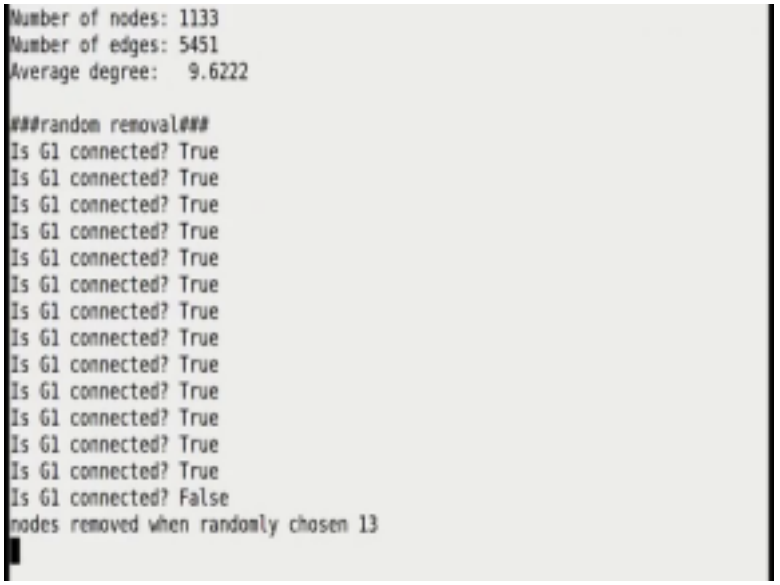
And again let me run this again.

(Refer Slide Time: 13:27)



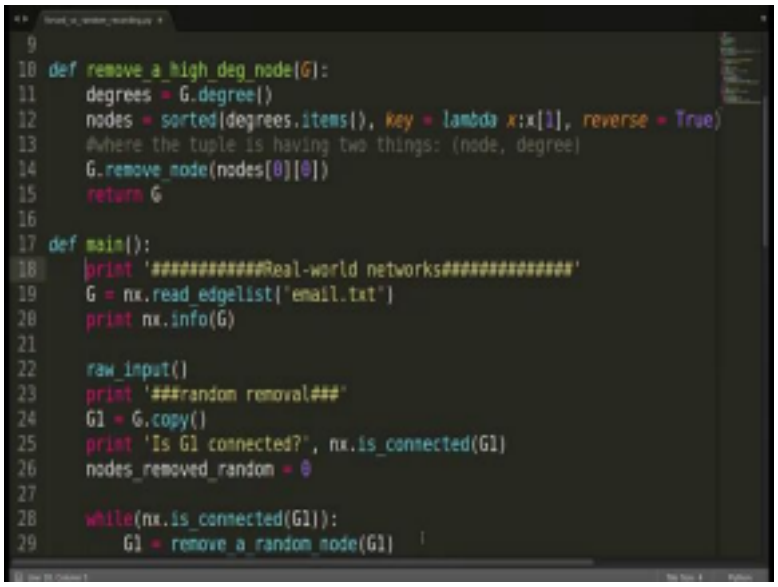
So, in this case you see randomly removal took 16 iterations and the selective removal took only 1.

(Refer Slide Time: 13:37)



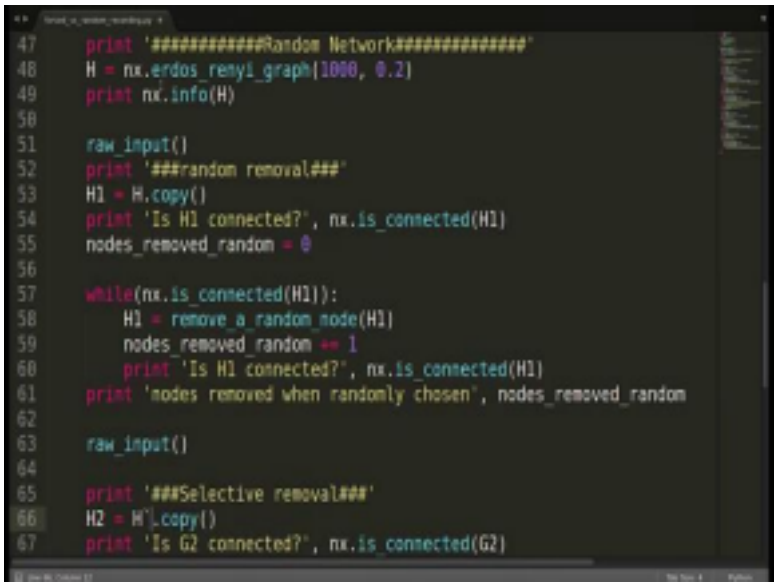
Let me run this again in random removal, it took 13 and in then I in the selective one it took 1, I am just going to run it last time. So, again in randomly it took 17 and in selective it took 1. You can just keep checking that ok. So, that is one thing. In case of real-world networks this sort of thing happens. Now, we are going to check the same thing on a random network ok.

(Refer Slide Time: 14:10)



So, let me let me write here that the all this was for all this was for real world network or a network which basically follows power law distribution and preferential attachment, real world networks right. This does not look good, let me write this does not, let me write this. So, this was for real world networks. This looks very bad anyway, let us go ahead.

(Refer Slide Time: 14:49)



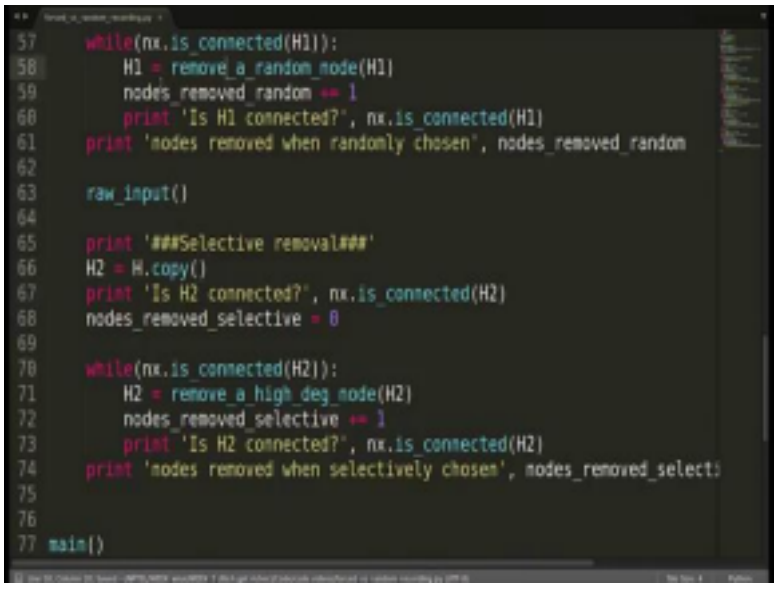
So, next we are going to do the same things for a random for a random network. Now how can we create a random network? Ok in the previous videos we created an Erdos

Renyi network. We can use that code to create a network and we can also use a function from network x where we have a function where we pass the values of n and p and we get a random network.

So, in order to tell you the function, I will use that function only over here. Let me call the graph H. I wrote nx.erdos_renyi network. So, this is a I am sorry, graph. So, this is the name of the function and let me pass some n, the total number of nodes and the value of t the probability. This should give us random network. Let me print the basic information of H.

After that we want to remove the nodes as the same way we did earlier. Let me copy paste the entire code here and we will make changes into that to this right. So, I am just pasting the entire thing here, random network starting here only. Raw input and then random removal l the graph name is H, what to write to ok. We are making copy of H and then we are checking whether H1 is connected or not and nodes removed is find and then nx is this connected H, (Refer Time: 16:47) I could have actually use here replace like replace function here, it is ok. So, everything else remains the same, I am just changing the graph here ok.

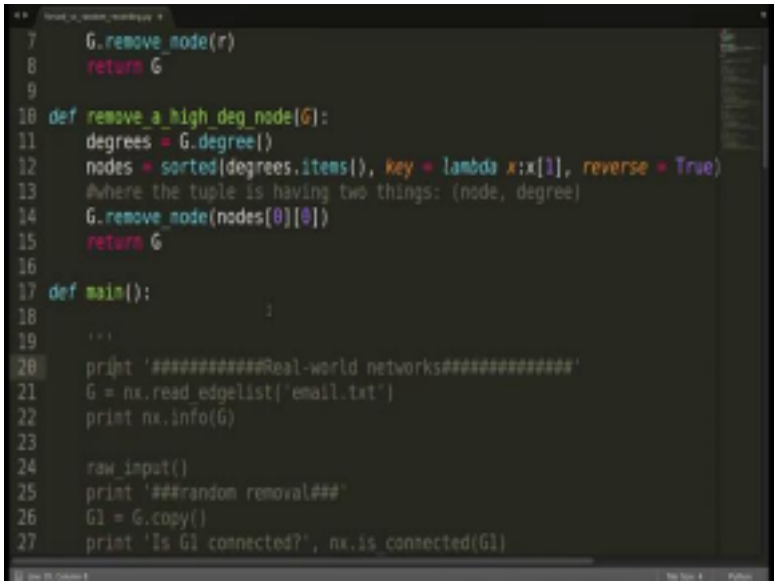
(Refer Slide Time: 17:00)



I actually should have use replace method, but I think we are done we will almost done now, check it last one alright. So, I think we have good to go. We have just taken random network now and we are applying random removal as well as selective remove in this

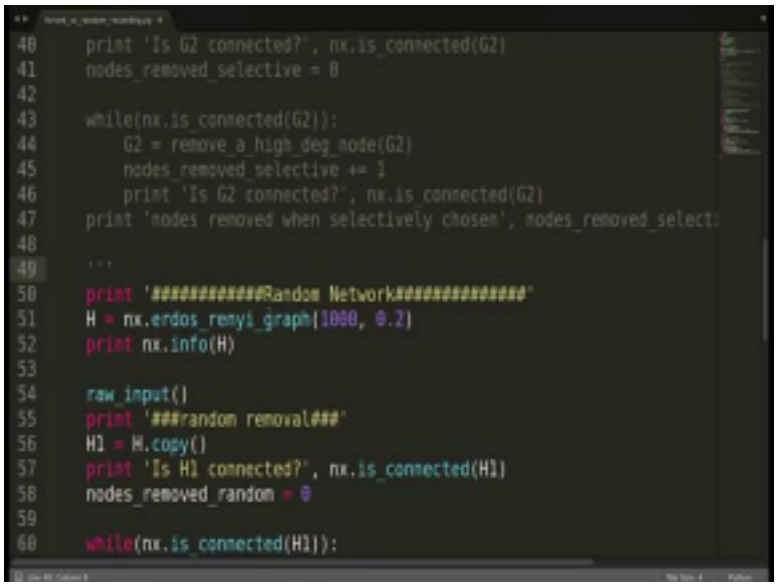
case. And we are continuing the number of iterations that are required in both the cases ok.

(Refer Slide Time: 17:39)



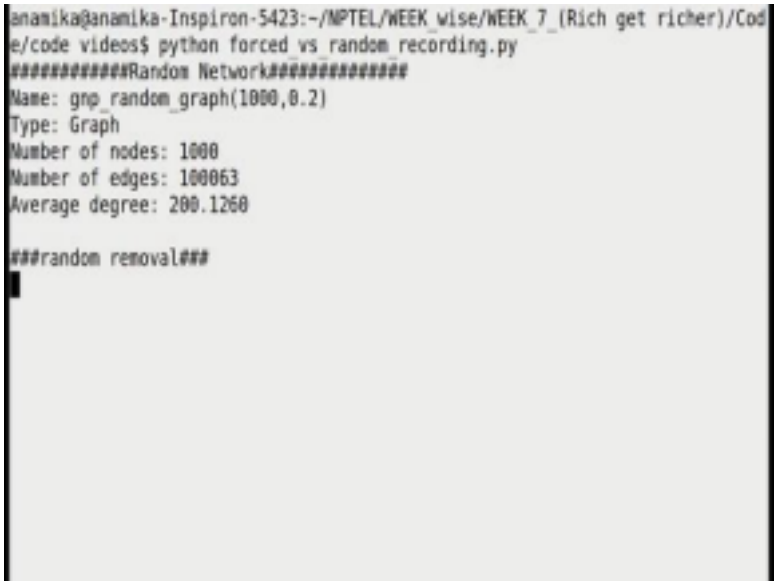
Since, we have already observed the behaviour on a real-world network, let me comment that part. So, that we are able to concentrate on only real-world network.

(Refer Slide Time: 17:45)



So, I am commenting this part up to which point this point ok.

(Refer Slide Time: 17:50)



So, let us check our code. So, this is for random network we have 1000 nodes and these many edges. I am pressing enter, random removal is taking place.

(Refer Slide Time: 18:01)



So, you see this is taking a lot of time it is taking a (Refer Time: 18:07) number of time. It is actually going on and on as you can see it is going on and on still going on ok. So, the number of nodes we had to remove from a random network when we randomly choose them was 985. So, (Refer Time: 18:28) 985.

I am pressing enter here. Now we are selectively removing that is we are choosing the nodes with high degree and then we are removing them. It is again going on and on

surprisingly it is going on and on; let us go on and on 928 as you can see. So, in the case of random network when we randomly it remove the nodes, it took a some 938 nodes to make the graph disconnected, and in case when we remove the nodes based on their degrees that is we remove the high degree nodes and every iteration, it again took as 928 nodes to make the graph disconnected.

Now that is happening because, there are no hubs in random networks which are connecting a lot of nodes. So, all the nodes are sort of equally likely because, equally likely in the sense of the number of nodes that they are connected to. So, there is no preference because, the edges were added randomly and there is no preference towards addition of nodes to some particular nodes in this.

So, that is a sort of difference that we observe in case of real-world networks as well as random networks, because real world networks are not random and they actually exhibit some sort of preference towards the nodes.